

Informe de Proyecto: Alke Web Base

Autor: Ignacio Trujillo

Módulo: Desarrollo Web con Django - Fundamentos

Herramientas utilizadas: Visual Studio Code, Django

Fecha: 15 de marzo de 2026

Proyecto Alke Web Base

Guía paso a paso y documento explicativo

Objetivo

El objetivo de este informe es detallar el proceso de desarrollo y la arquitectura técnica del proyecto realizado. Dado que el sistema se concibe como un entorno de experimentación y consolidación de conocimientos, este informe actúa como una guía metodológica paso a paso. El objetivo final es establecer un marco de referencia técnico que sirva de base para la implementación de futuras aplicaciones y proyectos escalables dentro del framework Django.

Estructura de trabajo

Etapas 1 — Configuración inicial del proyecto Django

1. Crear entorno virtual
2. Instalar Django
3. Crear proyecto base
4. Ejecutar servidor de desarrollo
5. Verificar funcionamiento inicial

Etapas 2 — Creación de la aplicación

1. Crear app principal
2. Registrar app en INSTALLED_APPS
3. Revisar estructura generada

Etapas 3 — Vistas y rutas

1. Crear vista inicial
2. Configurar urls.py del proyecto
3. Configurar urls.py de la app
4. Probar ruta funcional

Etapas 4 — Templates y archivos estáticos

1. Crear base.html
2. Crear página de inicio
3. Configurar carpeta templates
4. Configurar carpeta static

5. Aplicar CSS base

Etapas 5 — Formularios

1. Crear formulario básico o ModelForm
2. Integrar formulario en vista
3. Mostrar validaciones y mensajes

Etapas 6 — Autenticación y autorización

1. Configurar login/logout
2. Crear vista protegida
3. Definir redirecciones

1. Introducción

Este proyecto consiste en el desarrollo de una aplicación web básica en Django llamada **Alke Web Base**. Su propósito es demostrar la comprensión de la estructura del framework, el uso de rutas, vistas, templates, archivos estáticos, formularios, autenticación y panel administrativo.

2. Objetivo del proyecto

Desarrollar una aplicación web funcional en Django aplicando los fundamentos del framework y documentando su estructura, funcionamiento y evidencias de ejecución.

3. Tecnologías utilizadas

- Python
- Django
- Visual Studio Code
- HTML
- CSS
- SQLite

4. Estructura del proyecto

```
/Proyecto-Modulo-6-Desarrollo-Web-Django
├── db.sqlite3
├── manage.py
├── README.md
├── venv/
├── alke_web/
│   ├── __init__.py
│   ├── asgi.py
│   ├── settings.py
│   ├── urls.py
│   └── wsgi.py
├── webapp/
│   ├── migrations/
│   ├── static/
│   │   └── webapp/
│   │       └── style.css
│   ├── templates/
│   │   └── webapp/
│   │       ├── base.html
│   │       ├── inicio.html
│   │       ├── productos.html
│   │       ├── crear_producto.html
│   │       ├── login.html
│   │       └── dashboard.html
│   ├── __init__.py
│   ├── admin.py
│   ├── apps.py
│   ├── forms.py
│   ├── models.py
│   ├── test.py
│   ├── urls.py
│   └── views.py
```

5. Desarrollo paso a paso

5.1 Configuración inicial del proyecto Django

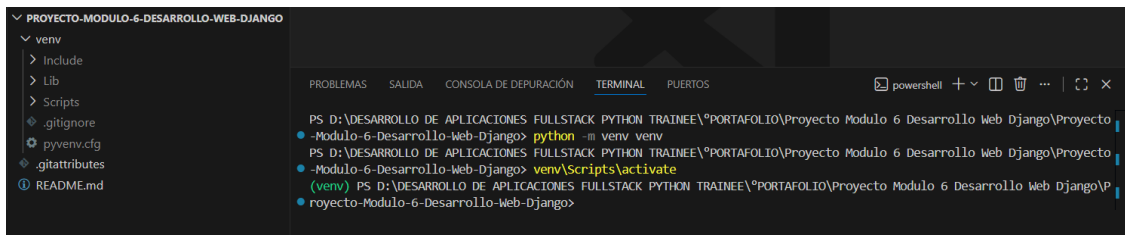
En esta etapa se preparó el entorno de desarrollo para la construcción del proyecto **Alke Web Base**. Primero se creó un entorno virtual con Python mediante el comando **“Python -m venv venv”** para aislar las dependencias del proyecto y evitar conflictos con otras instalaciones del sistema. Posteriormente, se instaló Django mediante **“pip install django”** y se verificó su correcta instalación consultando la versión disponible con el comando **“django-admin --version”**.

Luego se generó el proyecto base utilizando el comando **“django-admin startproject alke_web.”**, lo que creó la estructura inicial del proyecto, incluyendo archivos clave como manage.py, settings.py y urls.py. Finalmente, se ejecutó el servidor de desarrollo con **“python manage.py runserver”** y se verificó en el navegador que la aplicación funcionara correctamente mostrando la pantalla inicial de Django.

1. Creación y activación del entorno virtual

Comandos utilizados:

- **python -m venv venv**
- **venv\Scripts\activate**

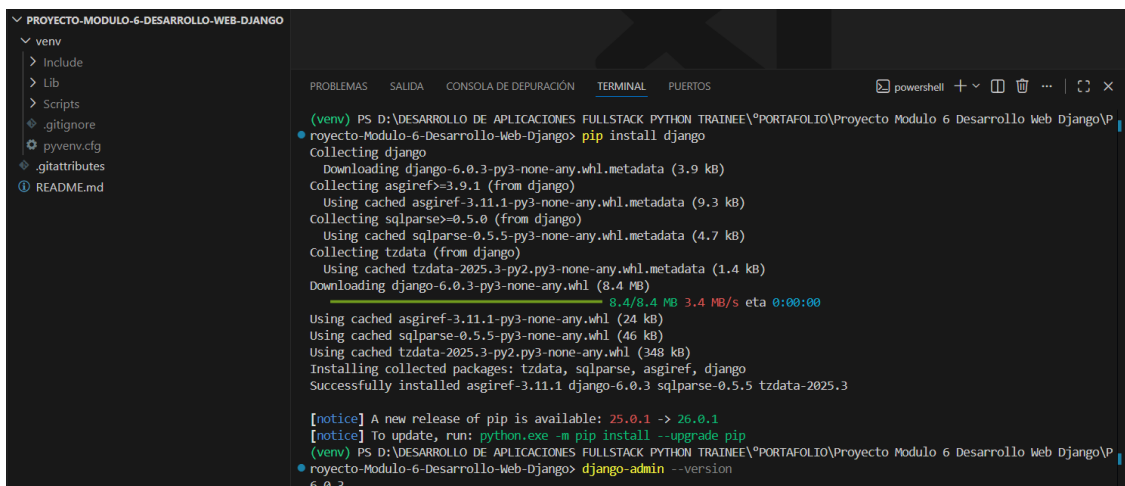


```
PS D:\DESARROLLO DE APLICACIONES FULLSTACK PYTHON TRAINEE\PORTAFOLIO\Proyecto Modulo 6 Desarrollo Web Django\Proyecto-Modulo-6-Desarrollo-Web-Django> python -m venv venv
PS D:\DESARROLLO DE APLICACIONES FULLSTACK PYTHON TRAINEE\PORTAFOLIO\Proyecto Modulo 6 Desarrollo Web Django\Proyecto-Modulo-6-Desarrollo-Web-Django> venv\Scripts\activate
(venv) PS D:\DESARROLLO DE APLICACIONES FULLSTACK PYTHON TRAINEE\PORTAFOLIO\Proyecto Modulo 6 Desarrollo Web Django\Proyecto-Modulo-6-Desarrollo-Web-Django>
```

2. Instalación de Django y verificación de versión instalada

Comandos utilizados:

- **pip install django**
- **django-admin --version**



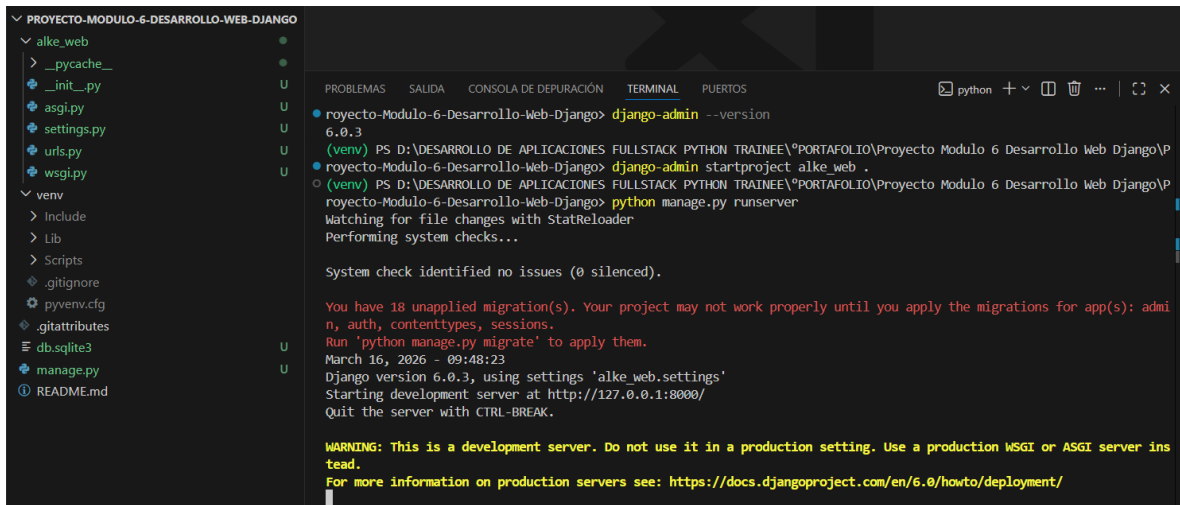
```
(venv) PS D:\DESARROLLO DE APLICACIONES FULLSTACK PYTHON TRAINEE\PORTAFOLIO\Proyecto Modulo 6 Desarrollo Web Django\Proyecto-Modulo-6-Desarrollo-Web-Django> pip install django
Collecting django
  Downloading django-6.0.3-py3-none-any.whl.metadata (3.9 kB)
Collecting asgiref<=3.9.1 (from django)
  Using cached asgiref-3.11.1-py3-none-any.whl.metadata (9.3 kB)
Collecting sqlparse>=0.5.0 (from django)
  Using cached sqlparse-0.5.5-py3-none-any.whl.metadata (4.7 kB)
Collecting tzdata (from django)
  Using cached tzdata-2025.3-py2.py3-none-any.whl.metadata (1.4 kB)
Downloading django-6.0.3-py3-none-any.whl (8.4 MB)
8.4/8.4 MB 3.4 MB/s eta 0:00:00
Using cached asgiref-3.11.1-py3-none-any.whl (24 kB)
Using cached sqlparse-0.5.5-py3-none-any.whl (46 kB)
Using cached tzdata-2025.3-py2.py3-none-any.whl (348 kB)
Installing collected packages: tzdata, sqlparse, asgiref, django
Successfully installed asgiref-3.11.1 django-6.0.3 sqlparse-0.5.5 tzdata-2025.3

[notice] A new release of pip is available: 25.0.1 -> 26.0.1
[notice] To update, run: python.exe -m pip install --upgrade pip
(venv) PS D:\DESARROLLO DE APLICACIONES FULLSTACK PYTHON TRAINEE\PORTAFOLIO\Proyecto Modulo 6 Desarrollo Web Django\Proyecto-Modulo-6-Desarrollo-Web-Django> django-admin --version
6.0.3
```

3. Creación del proyecto Alke_web y ejecución del servidor

Comandos utilizados

- `django-admin startproject alke_web .`
- `python manage.py runserver`



```
PROYECTO-MODULO-6-DESARROLLO-WEB-DJANGO
└─ alke_web
   ├── __pycache__
   ├── __init__.py
   ├── asgi.py
   ├── settings.py
   ├── urls.py
   └── wsgi.py
venv
├── Include
├── Lib
├── Scripts
├── .gitignore
├── pyvenv.cfg
├── .gitattributes
├── db.sqlite3
├── manage.py
└── README.md

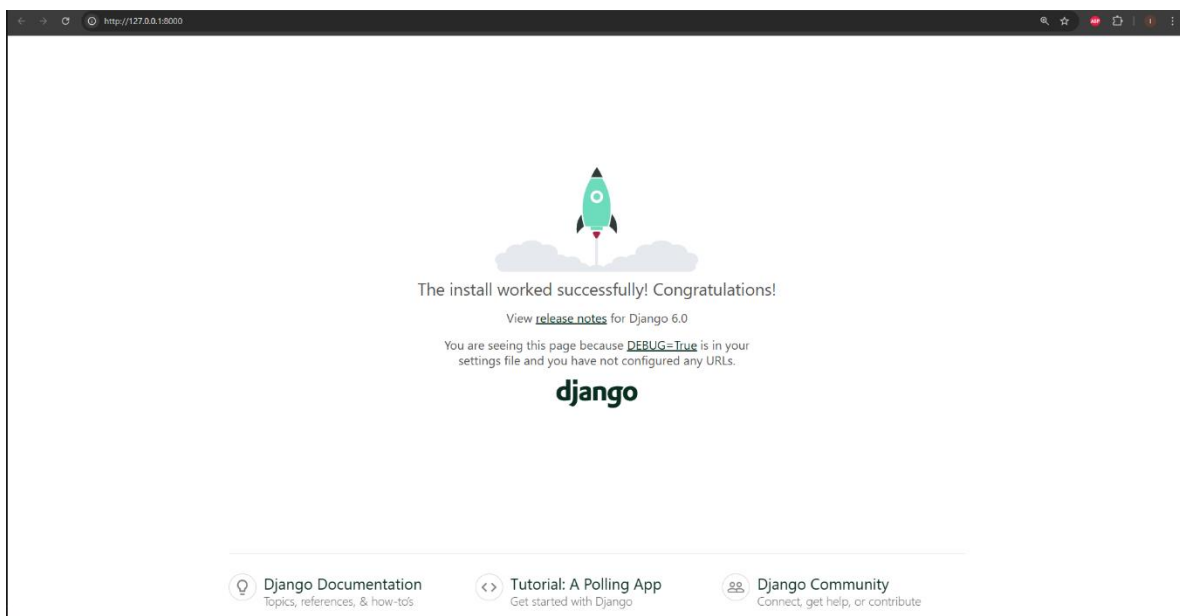
PROBLEMAS  SALIDA  CONSOLA DE DEPURACIÓN  TERMINAL  PUERTOS
royecto-Modulo-6-Desarrollo-Web-Django> django-admin --version
6.0.3
(venv) PS D:\DESARROLLO DE APLICACIONES FULLSTACK PYTHON TRAINEE\PORTAFOLIO\Proyecto Modulo 6 Desarrollo Web Django\P
royecto-Modulo-6-Desarrollo-Web-Django> django-admin startproject alke_web .
(venv) PS D:\DESARROLLO DE APLICACIONES FULLSTACK PYTHON TRAINEE\PORTAFOLIO\Proyecto Modulo 6 Desarrollo Web Django\P
royecto-Modulo-6-Desarrollo-Web-Django> python manage.py runserver
Watching for file changes with StatReloader
Performing system checks...

System check identified no issues (0 silenced).

You have 18 unapplied migration(s). Your project may not work properly until you apply the migrations for app(s): admin, auth, contenttypes, sessions.
Run 'python manage.py migrate' to apply them.
March 16, 2026 - 09:48:23
Django version 6.0.3, using settings 'alke_web.settings'
Starting development server at http://127.0.0.1:8000/
Quit the server with CTRL-BREAK.

WARNING: This is a development server. Do not use it in a production setting. Use a production WSGI or ASGI server instead.
For more information on production servers see: https://docs.djangoproject.com/en/6.0/howto/deployment/
```

4. Ejecución de servidor de Django y verificación en el navegador



5. Posteriormente se cierra el servidor para seguir con las modificaciones con “CTRL+C”

5.2 Creación de la aplicación

En esta etapa se creó la aplicación principal del proyecto utilizando el comando **“python manage.py startapp webapp”**. Esta aplicación funcionará como el módulo central donde se desarrollarán las vistas, modelos, formularios y configuraciones específicas de **Alke Web Base**.

La creación de la app generó automáticamente una estructura base compuesta por archivos como **“views.py”**, **“models.py”**, **“admin.py”** y **“apps.py”**, cada uno cumple con una función específica dentro de la arquitectura del proyecto. Posteriormente, la aplicación fue registrada en la lista **“INSTALLED_APPS”** del archivo **“settings.py”**, permitiendo que Django la reconociera como aplicación.

Finalmente, se volvió a ejecutar el servidor de desarrollo para comprobar que la integración de la nueva aplicación se hubiese realizado correctamente y sin errores.

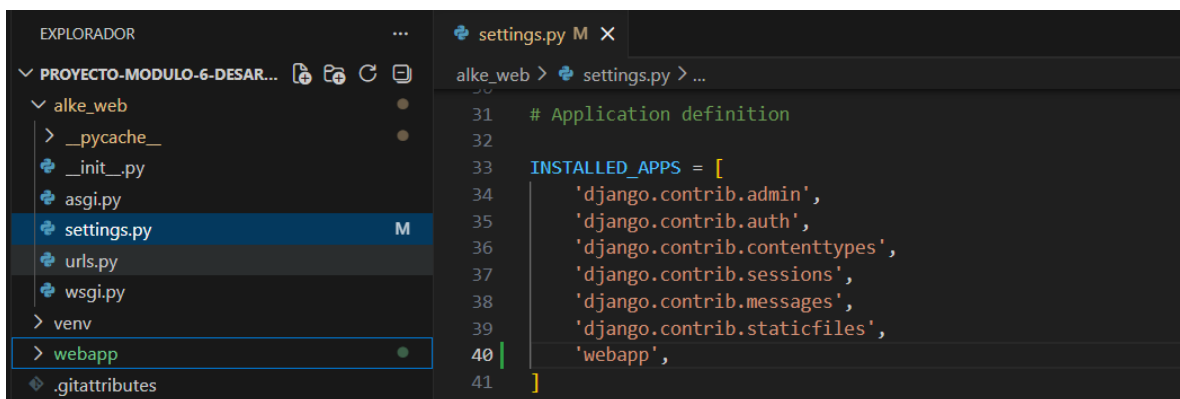
1. Creación de aplicación webapp

Comandos utilizados:

- **python manage.py startapp webapp**



2. Registro de la app “webapp” en “settings.py”



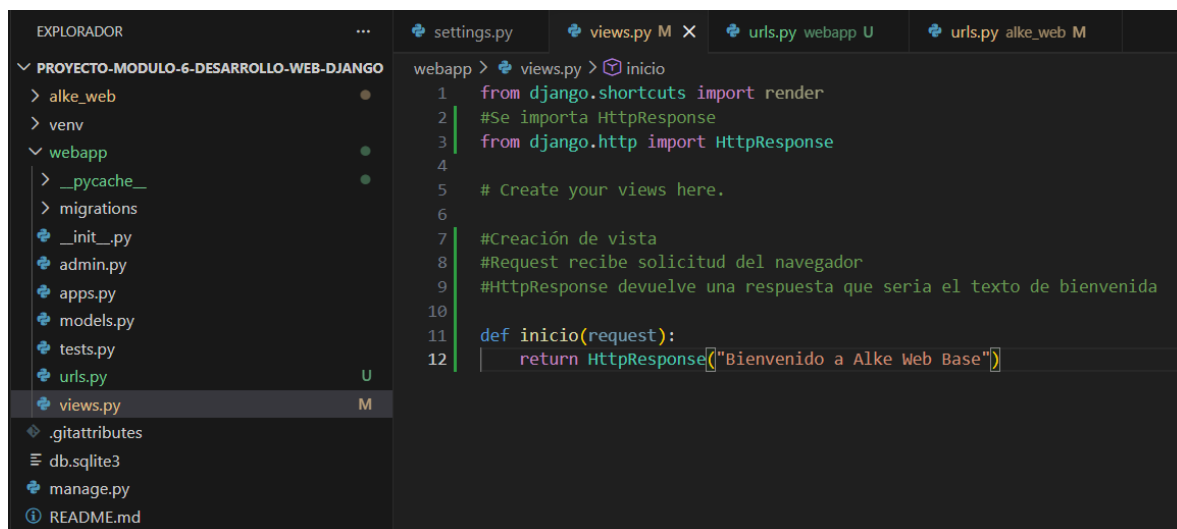
5.3 Configuración de vistas y rutas

En esta etapa se implementó la primera vista del proyecto. Para ello, en el archivo **“views.py”** de la aplicación principal **“webapp”** se creó una función llamada **“inicio”**, cuya finalidad fue responder a una solicitud HTTP devolviendo un mensaje de bienvenida.

Posteriormente, se creó el archivo **“urls.py”** dentro de la aplicación **“webapp”**, donde se definió la ruta principal de la vista. Luego, en el archivo **“urls.py”** del proyecto **“Alke Web Base”**, se integraron las rutas de la aplicación mediante la función **“include()”**, permitiendo que la ruta principal del sitio cargara el contenido definido en la app.

Finalmente, se ejecutó el servidor de desarrollo y se comprobó en el navegador que la ruta principal del proyecto mostrara correctamente el mensaje **“Bienvenido a Alke Web Base”**, validando así el flujo de funcionamiento entre URL, vista y respuesta.

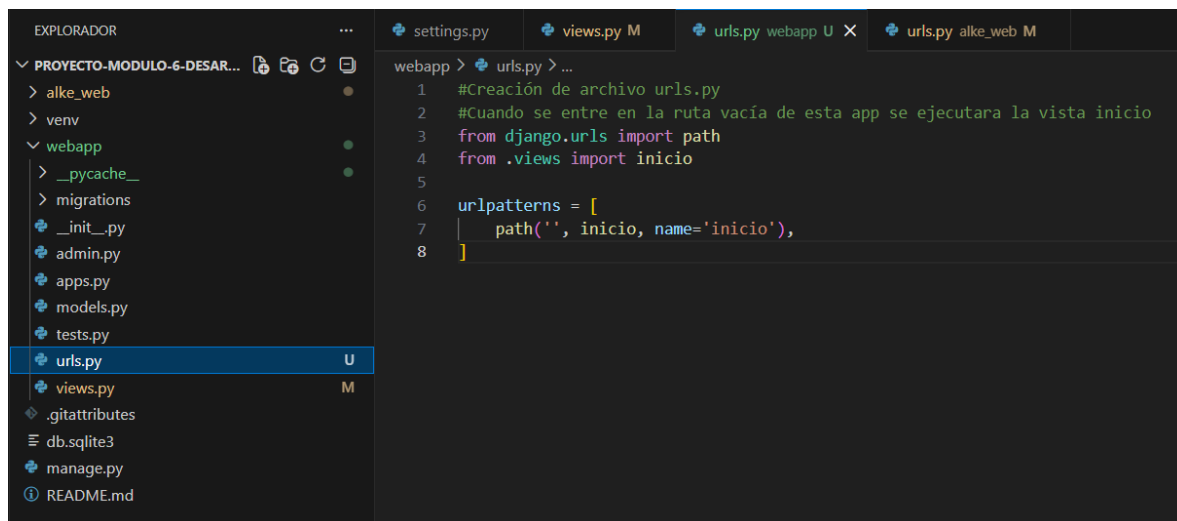
1. Creación de la vista en “webapp/views.py”.



The screenshot shows the Visual Studio Code interface. On the left, the Explorer sidebar displays the project structure under 'PROYECTO-MODULO-6-DESARROLLO-WEB-DJANGO'. The 'webapp' folder is expanded, showing files like __pycache__, migrations, __init__.py, admin.py, apps.py, models.py, tests.py, urls.py, and views.py. The 'views.py' file is selected and highlighted. The main editor area shows the code for 'views.py' with the following content:

```
webapp > views.py > inicio
1  from django.shortcuts import render
2  #Se importa HttpResponse
3  from django.http import HttpResponse
4
5  # Create your views here.
6
7  #Creación de vista
8  #Request recibe solicitud del navegador
9  #HttpResponse devuelve una respuesta que seria el texto de bienvenida
10
11 def inicio(request):
12     return HttpResponse("Bienvenido a Alke Web Base")
```

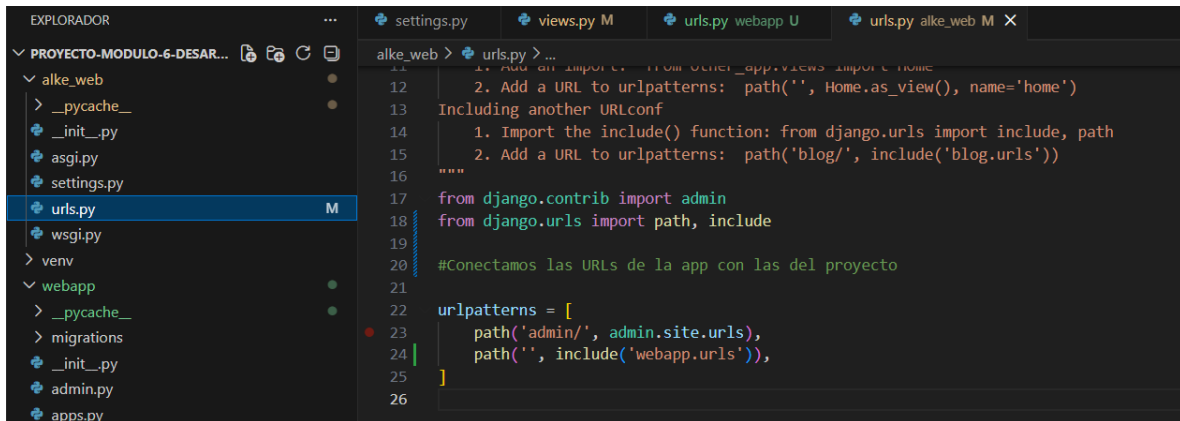
2. Creación de archivo urls.py dentro de la app webapp/



The screenshot shows the Visual Studio Code interface. On the left, the Explorer sidebar shows the same project structure as before, but now the 'urls.py' file in the 'webapp' folder is selected and highlighted. The main editor area shows the code for 'urls.py' with the following content:

```
webapp > urls.py > ...
1  #Creación de archivo urls.py
2  #Cuando se entre en la ruta vacía de esta app se ejecutara la vista inicio
3  from django.urls import path
4  from .views import inicio
5
6  urlpatterns = [
7      path('', inicio, name='inicio'),
8  ]
```

3. Conectar las URL's de la webapp con el proyecto alke_web

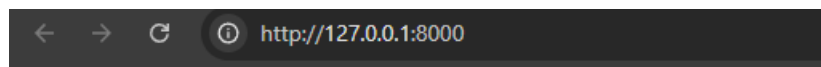


```
11 1. Add an import: from other_app.views import home
12 2. Add a URL to urlpatterns: path('', Home.as_view(), name='home')
13 Including another URLconf
14 1. Import the include() function: from django.urls import include, path
15 2. Add a URL to urlpatterns: path('blog/', include('blog.urls'))
16
17 from django.contrib import admin
18 from django.urls import path, include
19
20 #Conectamos las URLs de la app con las del proyecto
21
22 urlpatterns = [
23     path('admin/', admin.site.urls),
24     path('', include('webapp.urls')),
25 ]
26
```

4. Ejecución y verificación del servidor

Comando utilizado:

- `python manage.py runserver`



Bienvenido a Alke Web Base

5.4 Templates y archivos estáticos

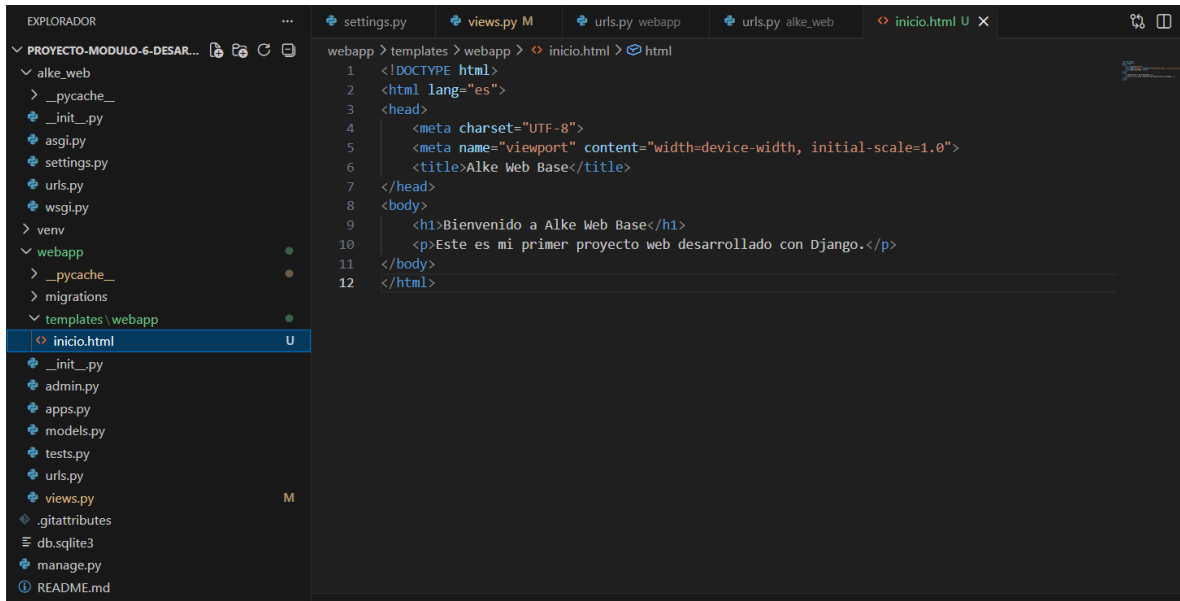
En esta etapa se implementó la primera interfaz visual del proyecto utilizando el sistema de templates de Django. Para ello, se creó una carpeta **“templates”** dentro de la aplicación principal, y en su interior un archivo **“inicio.html”**, el cual representa la página principal del sistema.

Luego, se modificó la vista inicio en **“views.py”** para utilizar la función **“render()”**, permitiendo devolver un template HTML en lugar de una respuesta de texto plano. Esto permitió que la aplicación comenzara a trabajar bajo el flujo típico de Django entre vista y plantilla.

Además, se creó la carpeta **“static”** para almacenar archivos estáticos del proyecto, específicamente una hoja de estilos CSS llamada **“style.css”**. Este archivo fue enlazado en la plantilla mediante la etiqueta **{% load static %}** y la función **{% static %}**, permitiendo aplicar diseño visual a la página principal.

Con esto, el proyecto pasó de mostrar solo una respuesta básica a presentar una página web estructurada, con contenido HTML y estilos personalizados, sentando la base visual para el desarrollo del resto del sistema.

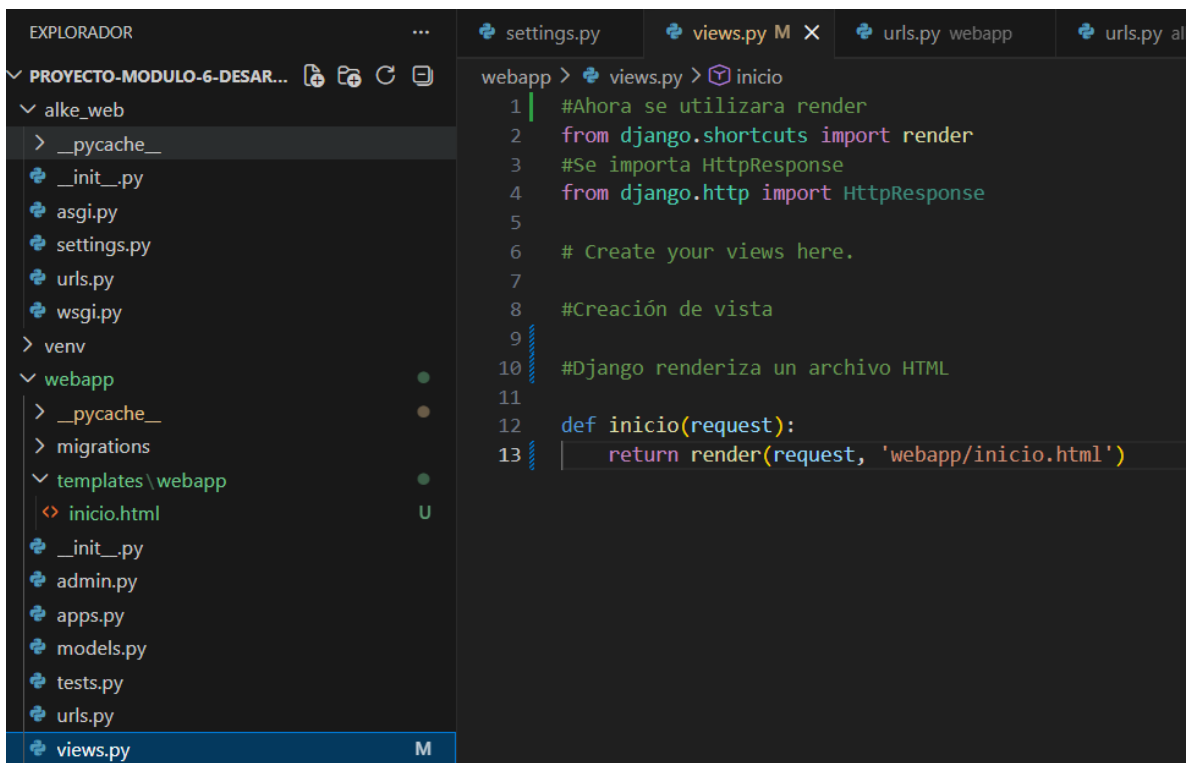
1. Creación del template “inicio.html” dentro del directorio de la aplicación “webapp/templates/webapp”



The screenshot shows the Visual Studio Code interface. On the left, the Explorer sidebar displays the project structure. The file `inicio.html` is highlighted under the path `webapp > templates > webapp`. The main editor window shows the content of `inicio.html`, which is an HTML template with the following code:

```
1 <!DOCTYPE html>
2 <html lang="es">
3 <head>
4   <meta charset="UTF-8">
5   <meta name="viewport" content="width=device-width, initial-scale=1.0">
6   <title>Alke Web Base</title>
7 </head>
8 <body>
9   <h1>Bienvenido a Alke Web Base</h1>
10  <p>Este es mi primer proyecto web desarrollado con Django.</p>
11 </body>
12 </html>
```

2. Modificación de la vista para usar “render”



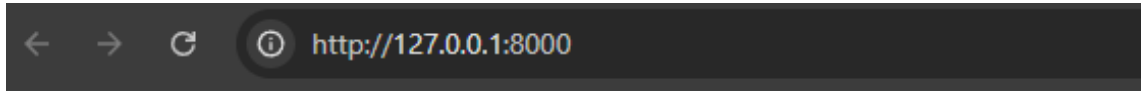
The screenshot shows the Visual Studio Code interface. The Explorer sidebar on the left shows the file `views.py` selected under the path `webapp > templates > webapp`. The main editor window shows the content of `views.py`, which has been modified to use the `render` function from `django.shortcuts`. The code is as follows:

```
1 #Ahora se utilizara render
2 from django.shortcuts import render
3 #Se importa HttpResponse
4 from django.http import HttpResponse
5
6 # Create your views here.
7
8 #Creación de vista
9
10 #Django renderiza un archivo HTML
11
12 def inicio(request):
13     return render(request, 'webapp/inicio.html')
```

3. Verificación en el navegador

Comando utilizado

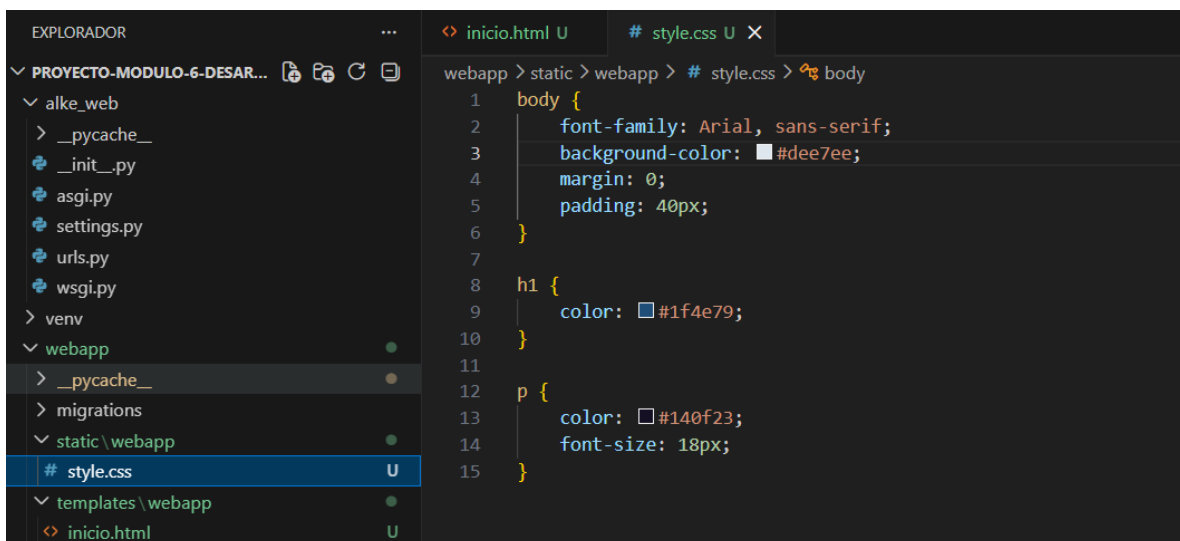
- `python manage.py runserver`



Bienvenido a Alke Web Base

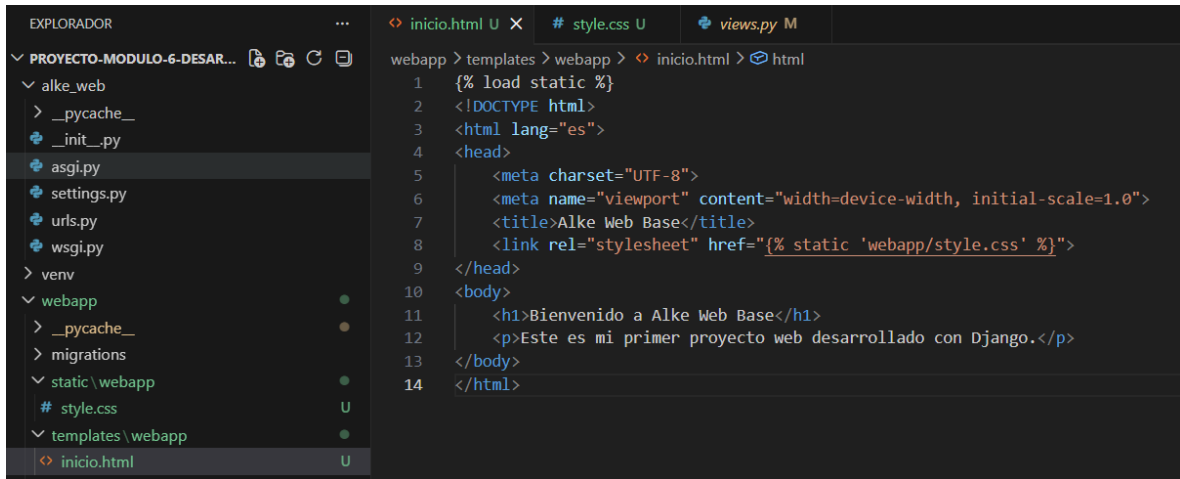
Este es mi primer proyecto web desarrollado con Django.

4. Creación del archivo CSS “style.css” dentro del directorio de la aplicación “webapp/static/webapp”



5. Modificación para cargar archivos estáticos en “inicio.html”

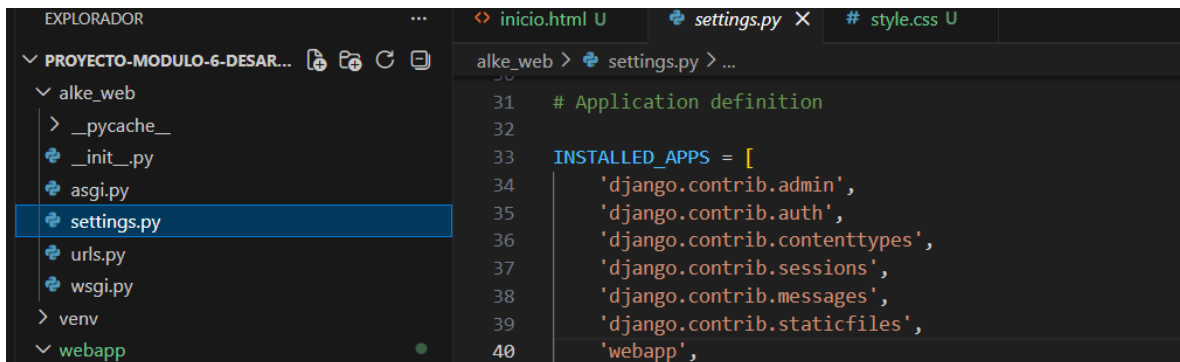
- {% load static %} habilita el uso de archivos estáticos
- {% static 'webapp/style.css' %} enlaza el archivo CSS



The screenshot shows the VS Code interface. On the left, the 'EXPLORADOR' (Explorer) sidebar shows the project structure: 'PROYECTO-MODULO-6-DESAR...' containing 'alke_web' (with subfolders '__pycache__', '_init_.py', 'asgi.py', 'settings.py', 'urls.py', 'wsgi.py', 'venv', and 'webapp') and 'static/webapp' (containing 'style.css'). The 'webapp' folder is expanded, showing 'templates/webapp' and 'inicio.html'. The 'inicio.html' file is selected and its content is displayed in the main editor. The code is a Django template that loads static files and includes a CSS link.

```
1 {% load static %}
2 <!DOCTYPE html>
3 <html lang="es">
4 <head>
5   <meta charset="UTF-8">
6   <meta name="viewport" content="width=device-width, initial-scale=1.0">
7   <title>Alke Web Base</title>
8   <link rel="stylesheet" href="{% static 'webapp/style.css' %}">
9 </head>
10 <body>
11   <h1>Bienvenido a Alke Web Base</h1>
12   <p>Este es mi primer proyecto web desarrollado con Django.</p>
13 </body>
14 </html>
```

6. Verificación del archivo “settings.py” del proyecto “alke_web”, este debe contener “django.contrib.staticfiles” dentro de “INSTALLED_APPS”



The screenshot shows the VS Code interface with the 'settings.py' file open in the main editor. The file explorer on the left shows the project structure, with 'settings.py' selected under the 'alke_web' folder. The code in 'settings.py' shows the 'INSTALLED_APPS' list, which includes 'django.contrib.staticfiles'.

```
31 # Application definition
32
33 INSTALLED_APPS = [
34     'django.contrib.admin',
35     'django.contrib.auth',
36     'django.contrib.contenttypes',
37     'django.contrib.sessions',
38     'django.contrib.messages',
39     'django.contrib.staticfiles',
40     'webapp',
```

7. Verificación de la aplicación de estilos

El navegador debe mostrar el estilo aplicado



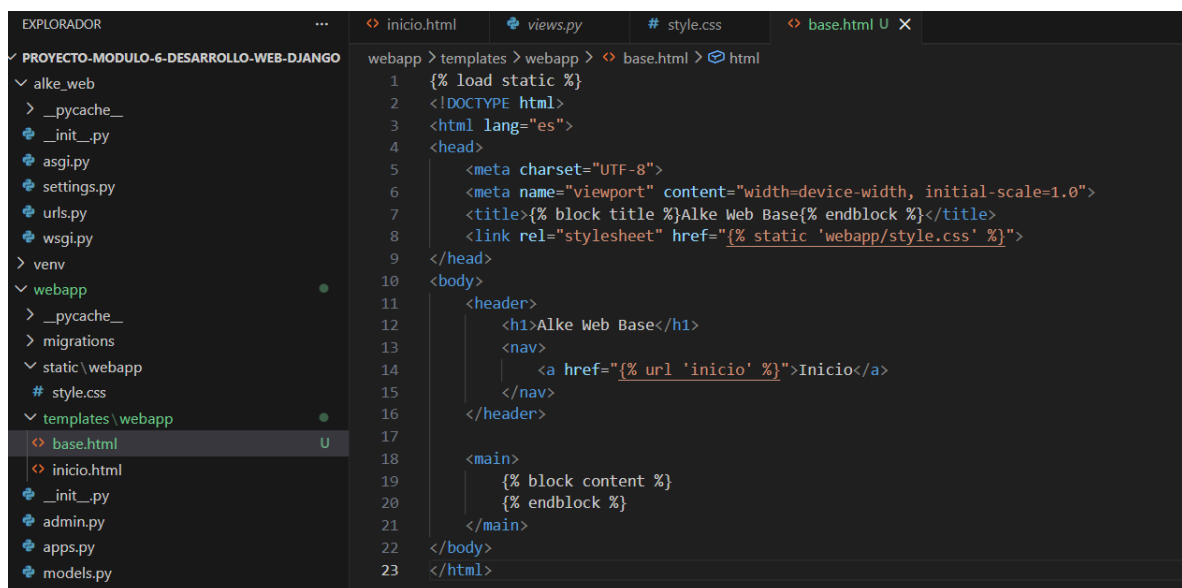
5.5 Estructura reutilizable con base.html

Con el objetivo de mejorar la organización del proyecto y evitar la duplicación de código HTML, se implementó una plantilla base llamada “base.html”. Esta plantilla se ubicó dentro de la carpeta “webapp/templates/webapp/” y se utilizó como estructura común del sitio, incorporando elementos reutilizables como el encabezado, el menú de navegación y la referencia a la hoja de estilos CSS.

Posteriormente, la plantilla “inicio.html” fue adaptada para heredar desde “base.html” mediante la etiqueta “{% extends %}”, utilizando bloques como “{% block title %}” y “{% block content %}” para insertar contenido específico en la página principal. Este enfoque permitió separar la estructura general del sitio del contenido particular de cada vista.

Además, se mejoró la hoja de estilos “style.css” para dar una apariencia más profesional al sistema, incorporando estilos para el encabezado, la navegación y el contenido principal. Esta implementación dejó preparada la base visual del proyecto para reutilizarse fácilmente en las futuras páginas de la aplicación.

1. Creación de archivo “base.html” en “webapp/templates/webapp/base.html”

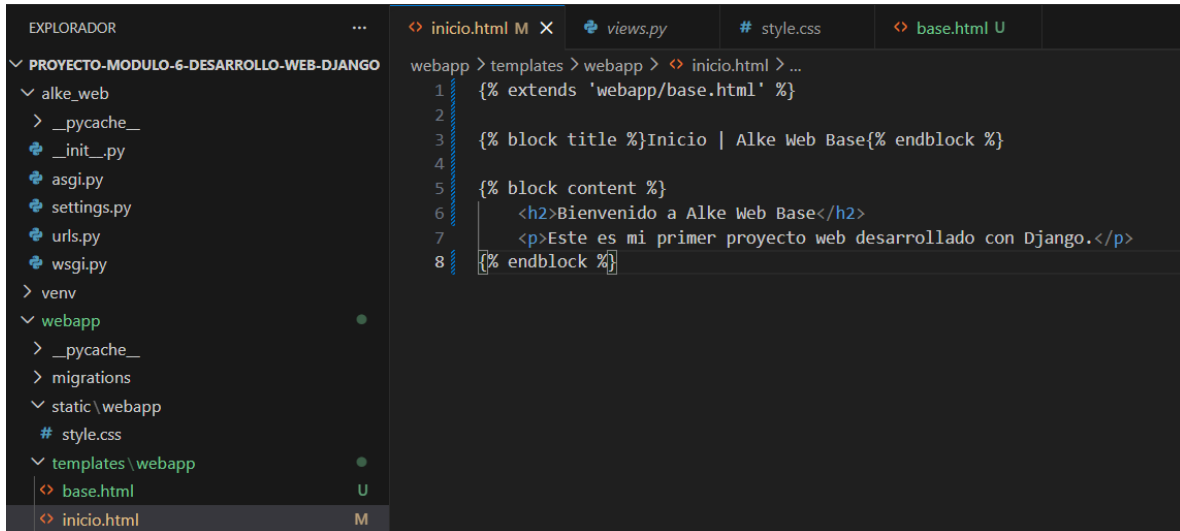


The screenshot shows a code editor with a file explorer on the left and a code editor on the right. The file explorer shows the project structure, with the file 'base.html' selected in the 'webapp/templates/webapp/' directory. The code editor shows the content of 'base.html'.

```
1 {% load static %}
2 <!DOCTYPE html>
3 <html lang="es">
4 <head>
5     <meta charset="UTF-8">
6     <meta name="viewport" content="width=device-width, initial-scale=1.0">
7     <title>{% block title %}Alke Web Base{% endblock %}</title>
8     <link rel="stylesheet" href="{% static 'webapp/style.css' %}">
9 </head>
10 <body>
11     <header>
12         <h1>Alke Web Base</h1>
13         <nav>
14             <a href="{% url 'inicio' %}">Inicio</a>
15         </nav>
16     </header>
17
18     <main>
19         {% block content %}
20         {% endblock %}
21     </main>
22 </body>
23 </html>
```

2. Modificación de inicio.html para que herede de base.html

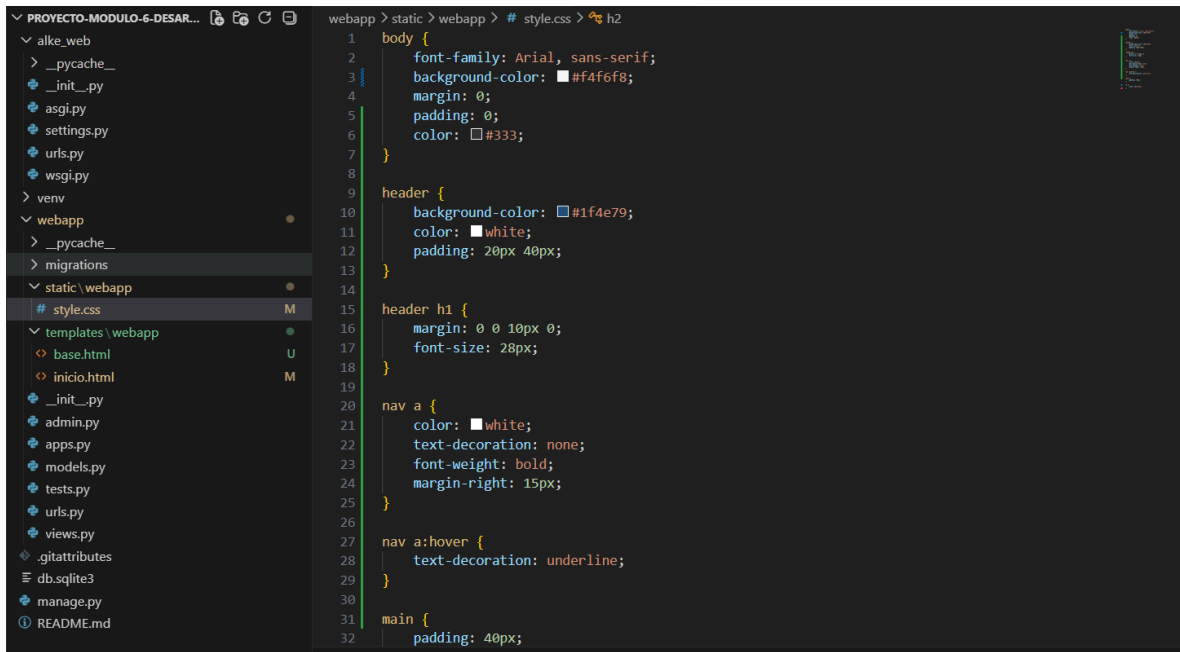
- {% extends 'webapp/base.html' %} hace que esta página use la estructura base
- {% block title %} cambia el título del navegador
- {% block content %} rellena el contenido principal de la página



The screenshot shows a code editor with a file explorer on the left and a code editor on the right. The file explorer shows the project structure, including the templates directory. The code editor shows the content of inicio.html, which has been modified to inherit from base.html.

```
webapp > templates > webapp > inicio.html > ...
1  {% extends 'webapp/base.html' %}
2
3  {% block title %}Inicio | Alke Web Base{% endblock %}
4
5  {% block content %}
6      <h2>Bienvenido a Alke Web Base</h2>
7      <p>Este es mi primer proyecto web desarrollado con Django.</p>
8  {% endblock %}
```

3. Aplicación de mejoras de estilos en archivo “style.css”



The screenshot shows a code editor with a file explorer on the left and a code editor on the right. The file explorer shows the project structure, including the static directory. The code editor shows the content of style.css, which has been modified to include styles for the web application.

```
webapp > static > webapp > # style.css > h2
1  body {
2      font-family: Arial, sans-serif;
3      background-color: #f4f6f8;
4      margin: 0;
5      padding: 0;
6      color: #333;
7  }
8
9  header {
10     background-color: #1f4e79;
11     color: white;
12     padding: 20px 40px;
13 }
14
15 header h1 {
16     margin: 0 0 10px 0;
17     font-size: 28px;
18 }
19
20 nav a {
21     color: white;
22     text-decoration: none;
23     font-weight: bold;
24     margin-right: 15px;
25 }
26
27 nav a:hover {
28     text-decoration: underline;
29 }
30
31 main {
32     padding: 40px;
```

4. Verificación en el navegador

Comando utilizado:

- `python manage.py runserver`



5.6 Modelo de datos y visualización dinámica de productos

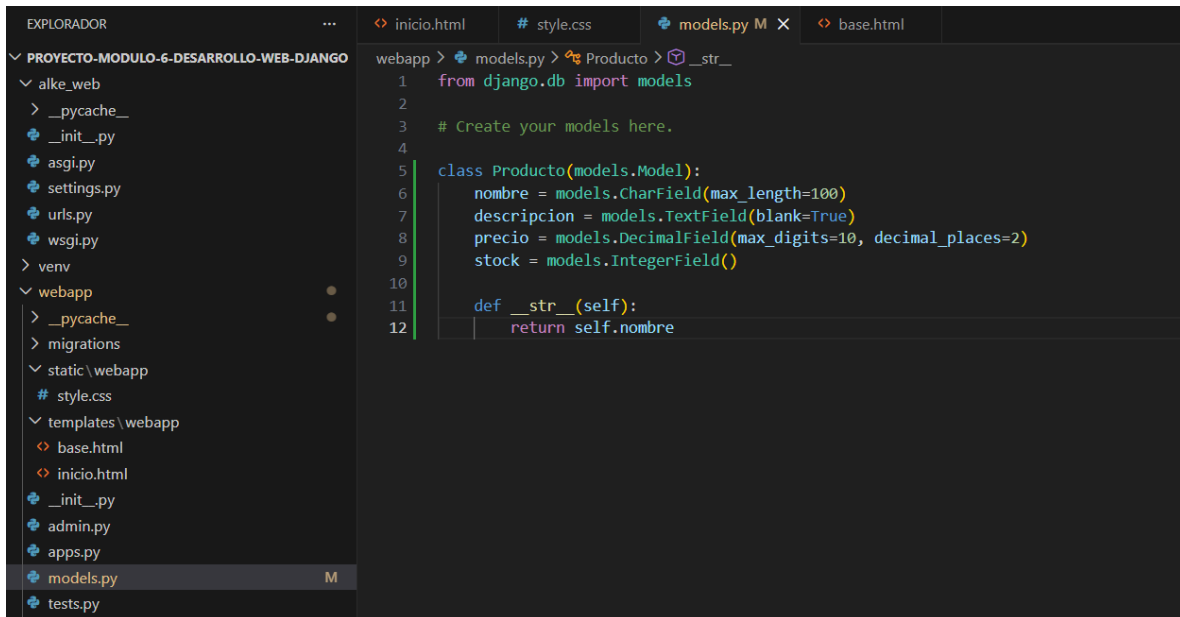
En esta etapa se implementó un modelo de datos llamado “Producto”, definido en el archivo “models.py” de la aplicación principal. Este modelo incluyó campos como nombre, descripción, precio y stock, permitiendo representar información básica de productos dentro de la base de datos del sistema.

Una vez definido el modelo, se ejecutaron los comandos “makemigrations” y “migrate” para generar y aplicar las migraciones necesarias, creando así la tabla correspondiente en la base de datos. Posteriormente, el modelo fue registrado en el archivo “admin.py”, lo que permitió administrarlo desde el panel de Django. Además, se personalizó su visualización utilizando “ModelAdmin”, incorporando columnas, filtros y herramientas de búsqueda.

Luego, se creó una vista llamada “lista_productos”, cuya función fue consultar todos los registros del modelo “Producto” mediante el ORM de Django y enviarlos a un template HTML. Para mostrar la información, se diseñó la plantilla “productos.html”, la cual heredó de “base.html” y utilizó estructuras dinámicas del motor de plantillas de Django como `{% for %}` y `{% if %}`.

Finalmente, se configuró la ruta “/productos/” en el archivo “urls.py” de la aplicación y se agregó un enlace de navegación en la plantilla base. Esto permitió visualizar en el navegador una lista dinámica de productos cargados desde la base de datos, consolidando el flujo completo entre modelo, vista, template y respuesta al usuario.

1. Creación de modelo “producto”



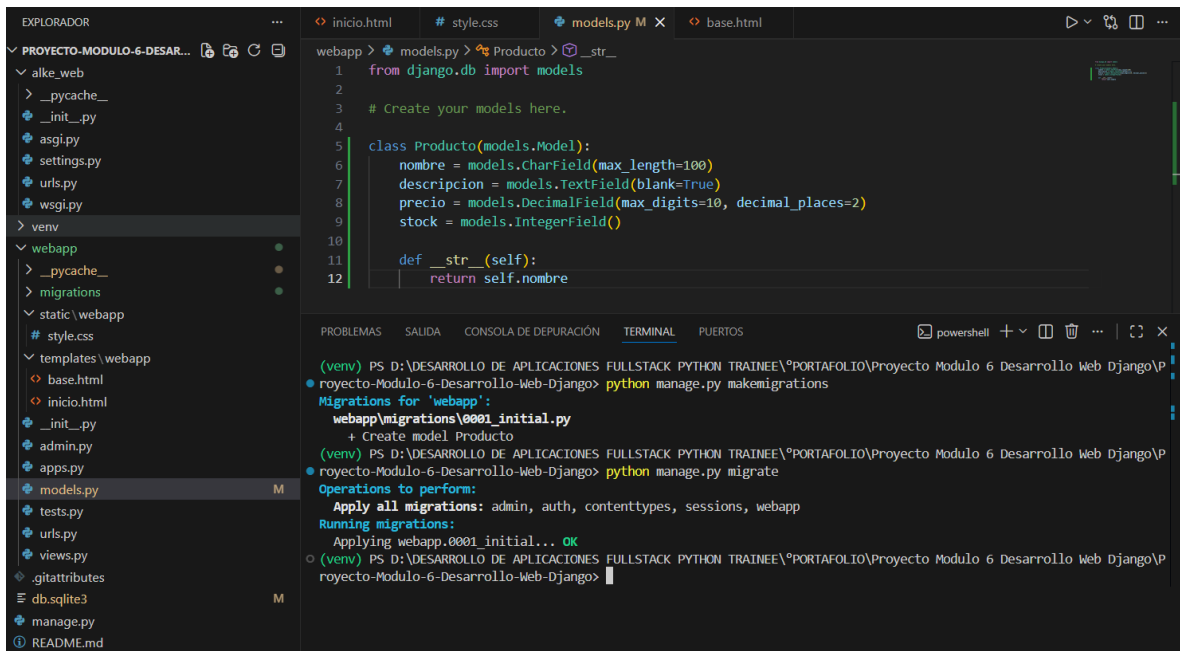
The screenshot shows the Visual Studio Code editor with a file explorer on the left and a code editor on the right. The file explorer shows a project structure for 'PROYECTO-MODULO-6-DESARROLLO-WEB-DJANGO'. The code editor shows the 'models.py' file with the following code:

```
webapp > models.py > Producto > __str__  
1 from django.db import models  
2  
3 # Create your models here.  
4  
5 class Producto(models.Model):  
6     nombre = models.CharField(max_length=100)  
7     descripcion = models.TextField(blank=True)  
8     precio = models.DecimalField(max_digits=10, decimal_places=2)  
9     stock = models.IntegerField()  
10  
11     def __str__(self):  
12         return self.nombre
```

2. Creación de migraciones luego de crear el modelo “producto”

Comando utilizado

- python manage.py makemigrations
- python manage.py migrate

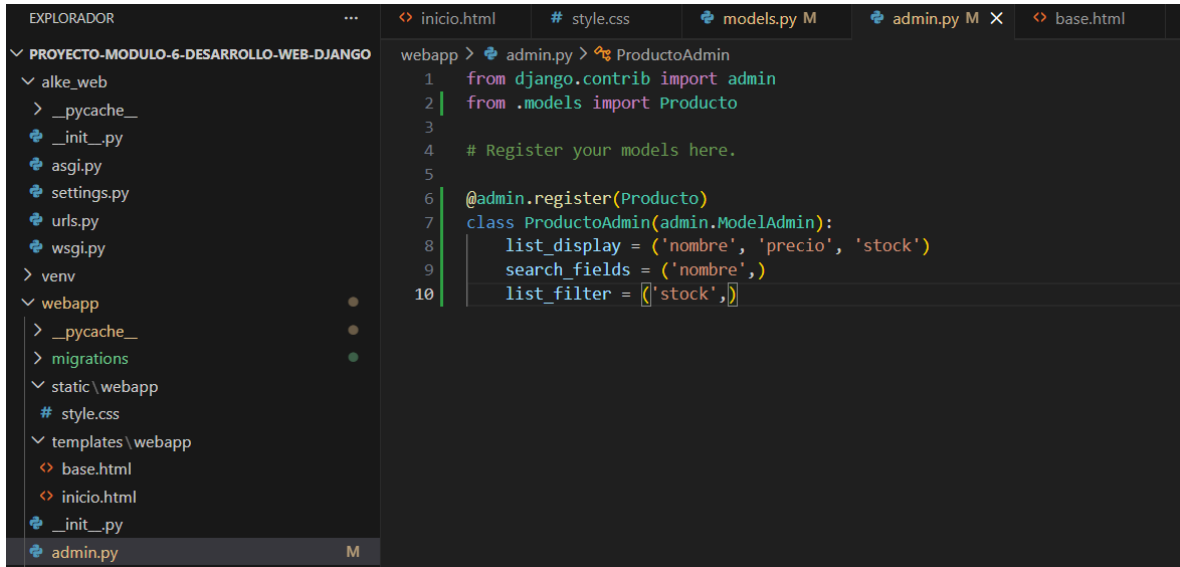


The screenshot shows the Visual Studio Code editor with the same file explorer and code editor as before. The terminal window at the bottom shows the execution of the following commands:

```
(venv) PS D:\DESARROLLO DE APLICACIONES FULLSTACK PYTHON TRAINEE\PORTAFOLIO\Proyecto Modulo 6 Desarrollo Web Django\Proyecto-Modulo-6-Desarrollo-Web-Django> python manage.py makemigrations  
Migrations for 'webapp':  
  webapp\migrations\0001_initial.py  
    + Create model Producto  
(venv) PS D:\DESARROLLO DE APLICACIONES FULLSTACK PYTHON TRAINEE\PORTAFOLIO\Proyecto Modulo 6 Desarrollo Web Django\Proyecto-Modulo-6-Desarrollo-Web-Django> python manage.py migrate  
Operations to perform:  
  Apply all migrations: admin, auth, contenttypes, sessions, webapp  
Running migrations:  
  Applying webapp.0001_initial... OK  
(venv) PS D:\DESARROLLO DE APLICACIONES FULLSTACK PYTHON TRAINEE\PORTAFOLIO\Proyecto Modulo 6 Desarrollo Web Django\Proyecto-Modulo-6-Desarrollo-Web-Django>
```

3. Registro del modelo en el panel admin

- `list_display`: muestra columnas en el admin
- `search_fields`: agrega buscador
- `list_filter`: agrega filtros laterales



The screenshot shows a code editor with a file explorer on the left and a code editor on the right. The file explorer shows a project structure for 'PROYECTO-MODULO-6-DESARROLLO-WEB-DJANGO'. The code editor shows the 'admin.py' file with the following code:

```
webapp > admin.py > % ProductoAdmin
1 from django.contrib import admin
2 from .models import Producto
3
4 # Register your models here.
5
6 @admin.register(Producto)
7 class ProductoAdmin(admin.ModelAdmin):
8     list_display = ('nombre', 'precio', 'stock')
9     search_fields = ('nombre',)
10    list_filter = ('stock',)
```

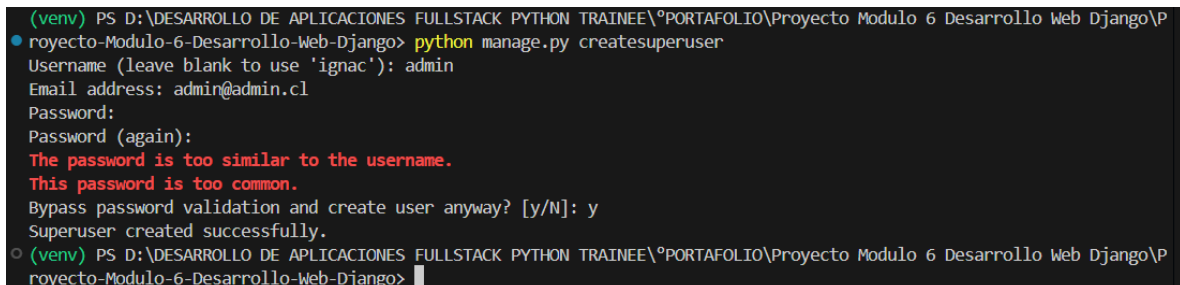
4. Creación de superusuario

Comandos utilizados:

- `python manage.py createsuperuser`

Características del usuario creado

- Username: admin
- Email address: admin@admin.cl
- Password: admin1234



The screenshot shows a terminal window with the following output:

```
(venv) PS D:\DESARROLLO DE APLICACIONES FULLSTACK PYTHON TRAINEE\PORTAFOLIO\Proyecto Modulo 6 Desarrollo Web Django\Proyecto-Modulo-6-Desarrollo-Web-Django> python manage.py createsuperuser
Username (leave blank to use 'ignac'): admin
Email address: admin@admin.cl
Password:
Password (again):
The password is too similar to the username.
This password is too common.
Bypass password validation and create user anyway? [y/N]: y
Superuser created successfully.
(venv) PS D:\DESARROLLO DE APLICACIONES FULLSTACK PYTHON TRAINEE\PORTAFOLIO\Proyecto Modulo 6 Desarrollo Web Django\Proyecto-Modulo-6-Desarrollo-Web-Django>
```

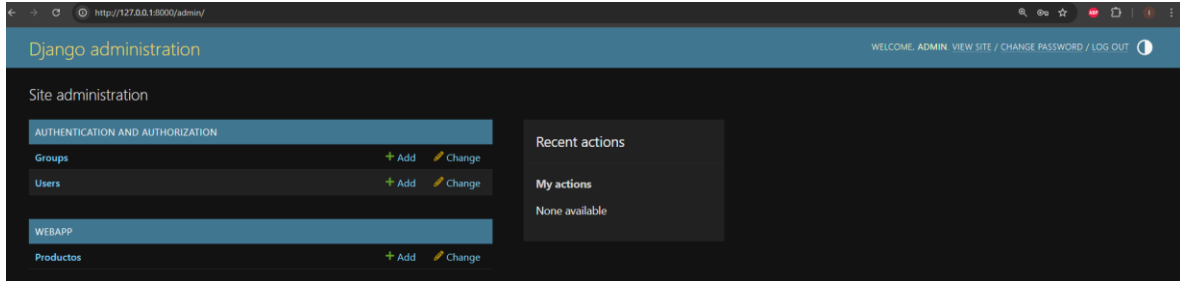
5. Carga de productos desde el admin

Comando utilizado:

- `python manage.py runserver`

Entrar a:

- <http://127.0.0.1:8000/admin/>



Productos de prueba ingresados

Producto 1	Producto 2	Producto 3
nombre: Notebook Lenovo	nombre: Mouse inalámbrico	nombre: Monitor Samsung
descripción: Equipo portátil para bootcamp	descripción: Mouse ergonómico recargable para bootcamp	descripción: Monitor Full HD 24 pulgadas para bootcamp
precio: 499990	precio: 15990	precio: 129990
stock: 8	stock: 15	stock: 6

Formulario para ingreso de productos

Django administration WELCOME, ADMIN. [VIEW SITE](#) / [CHANGE PASSWORD](#) / [LOG OUT](#)

[Home](#) > [Webapp](#) > [Productos](#) > Add producto

Start typing to filter...

AUTHENTICATION AND AUTHORIZATION

- [Groups](#) [+ Add](#)
- [Users](#) [+ Add](#)

WEBAPP

- [Productos](#) [+ Add](#)

Add producto

Nombre:

Descripcion:

Precio:

Stock:

[SAVE](#) [Save and add another](#) [Save and continue editing](#)

Productos ya ingresados

Django administration WELCOME, ADMIN. [VIEW SITE](#) / [CHANGE PASSWORD](#) / [LOG OUT](#)

[Home](#) > [Webapp](#) > [Productos](#)

Start typing to filter...

AUTHENTICATION AND AUTHORIZATION

- [Groups](#) [+ Add](#)
- [Users](#) [+ Add](#)

WEBAPP

- [Productos](#) [+ Add](#)

Select product to change

[Search](#)

Action: [-----](#) [Run](#) 0 of 3 selected

☐	NOMBRE	PRECIO	STOCK
☐	Monitor Samsung	129990.00	6
☐	Mouse Inalámbrico	15990.00	15
☐	Notebook Lenovo	499990.00	8

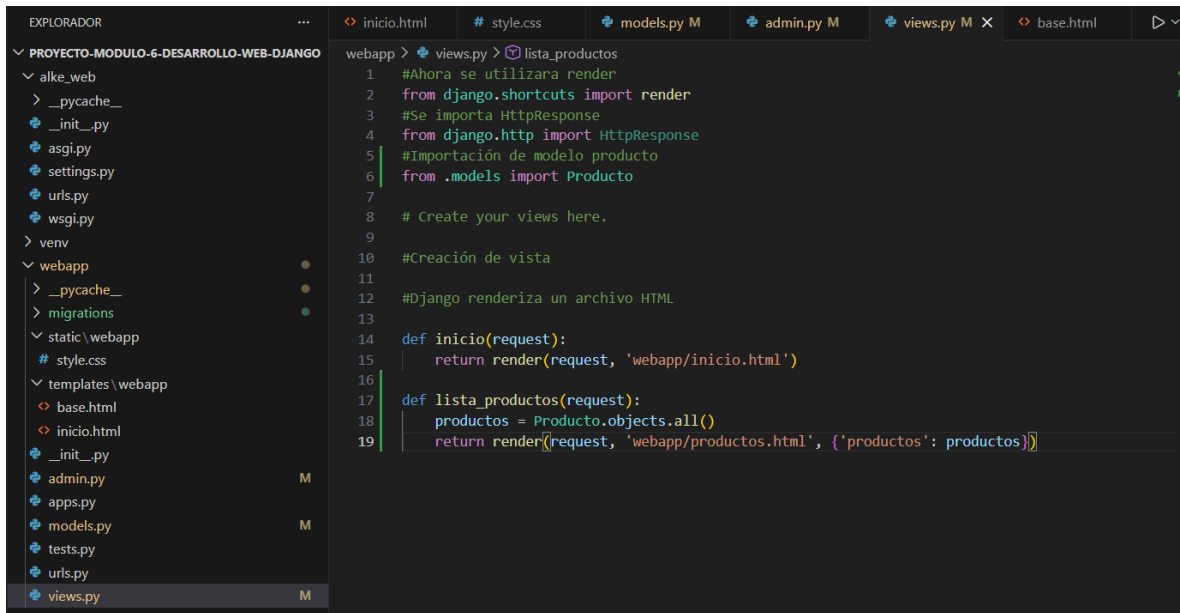
3 productos

[ADD PRODUCTO +](#)

FILTER

- [Show counts](#)
- 1 By stock
- [All](#)
- [6](#)
- [8](#)
- [15](#)

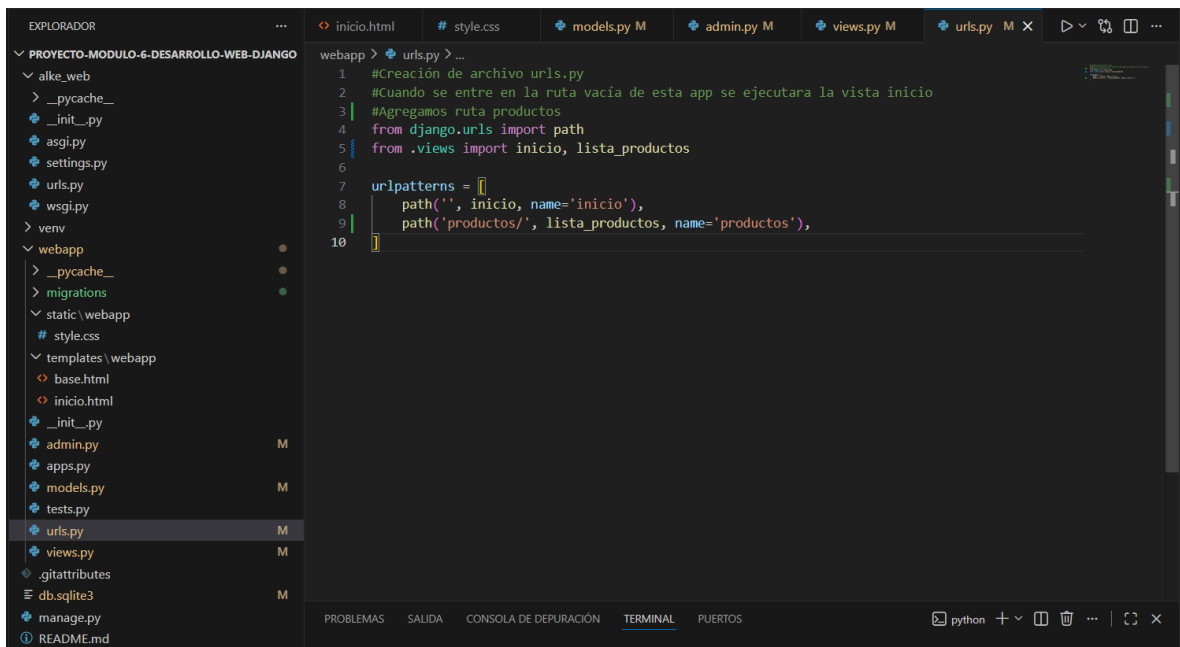
6. Creación de vista de productos “webapp/productos.html”



The screenshot shows the Visual Studio Code editor with a project named "PROYECTO-MODULO-6-DESARROLLO-WEB-DJANGO". The file explorer on the left shows the project structure, including the "webapp" directory. The main editor window displays the "views.py" file, which contains the following code:

```
webapp > views.py > lista_productos
1 #Ahora se utilizara render
2 from django.shortcuts import render
3 #Se importa HttpResponse
4 from django.http import HttpResponse
5 #Importación de modelo producto
6 from .models import Producto
7
8 # Create your views here.
9
10 #Creación de vista
11
12 #Django renderiza un archivo HTML
13
14 def inicio(request):
15     return render(request, 'webapp/inicio.html')
16
17 def lista_productos(request):
18     productos = Producto.objects.all()
19     return render(request, 'webapp/productos.html', {'productos': productos})
```

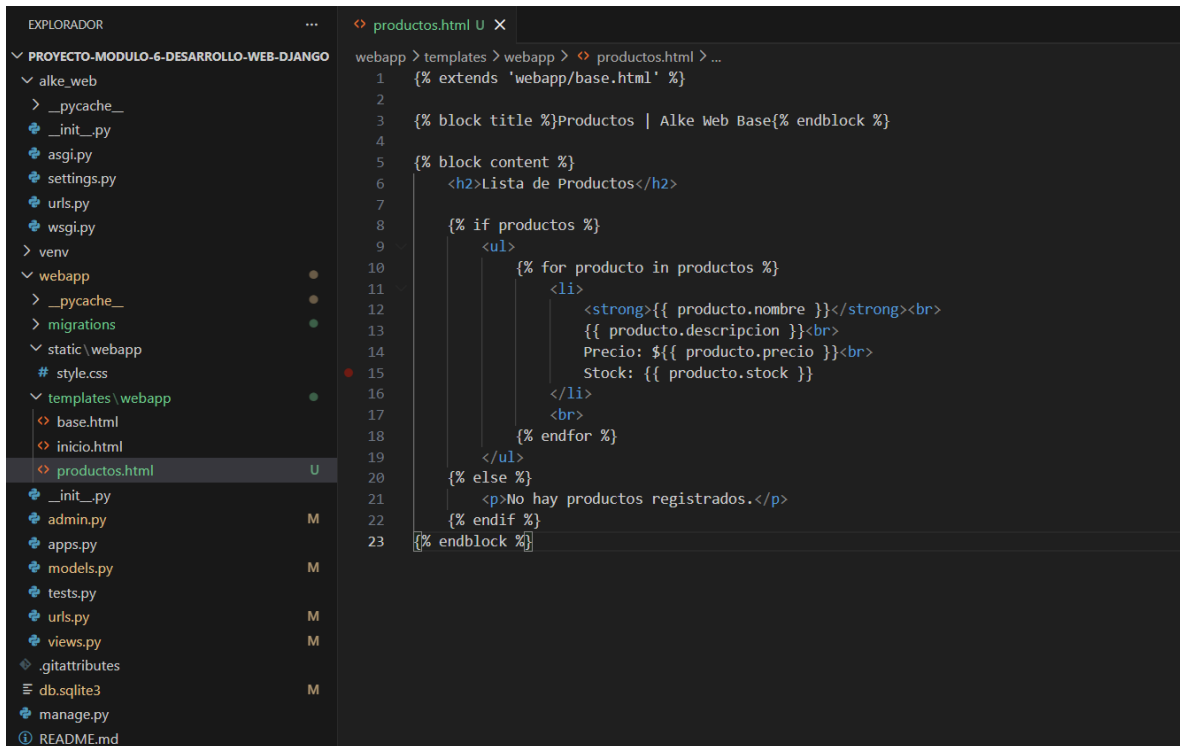
7. Incorporación de ruta de productos



The screenshot shows the Visual Studio Code editor with the same project. The file explorer on the left shows the project structure, including the "webapp" directory. The main editor window displays the "urls.py" file, which contains the following code:

```
webapp > urls.py > ...
1 #Creación de archivo urls.py
2 #Cuando se entre en la ruta vacia de esta app se ejecutara la vista inicio
3 #Agregamos ruta productos
4 from django.urls import path
5 from .views import inicio, lista_productos
6
7 urlpatterns = []
8     path('', inicio, name='inicio'),
9     path('productos/', lista_productos, name='productos'),
10 ]
```

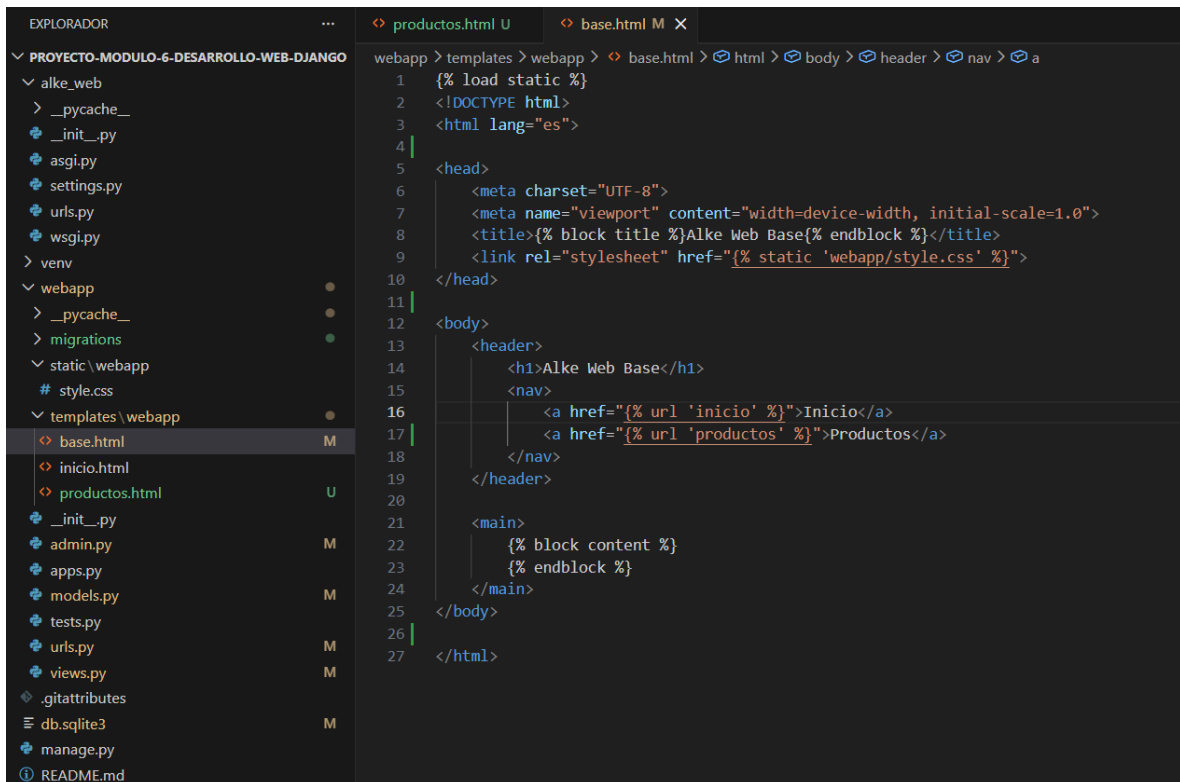
8. Creación del template “productos.html”



```
EXPLORADOR
PROYECTO-MODULO-6-DESARROLLO-WEB-DJANGO
  alke_web
    __pycache__
    __init__.py
    asgi.py
    settings.py
    urls.py
    wsgi.py
  venv
  webapp
    __pycache__
    migrations
    static\webapp
      style.css
    templates\webapp
      base.html
      inicio.html
      productos.html
    __init__.py
    admin.py
    apps.py
    models.py
    tests.py
    urls.py
    views.py
  .gitattributes
  db.sqlite3
  manage.py
  README.md

productos.html U
webapp > templates > webapp > productos.html > ...
1 {% extends 'webapp/base.html' %}
2
3 {% block title %}Productos | Alke Web Base{% endblock %}
4
5 {% block content %}
6   <h2>Lista de Productos</h2>
7
8   {% if productos %}
9     <ul>
10       {% for producto in productos %}
11         <li>
12           <strong>{{ producto.nombre }}</strong><br>
13           {{ producto.descripcion }}<br>
14           Precio: ${{ producto.precio }}<br>
15           Stock: {{ producto.stock }}
16         </li>
17       <br>
18     {% endfor %}
19   </ul>
20   {% else %}
21     <p>No hay productos registrados.</p>
22   {% endif %}
23 {% endblock %}
```

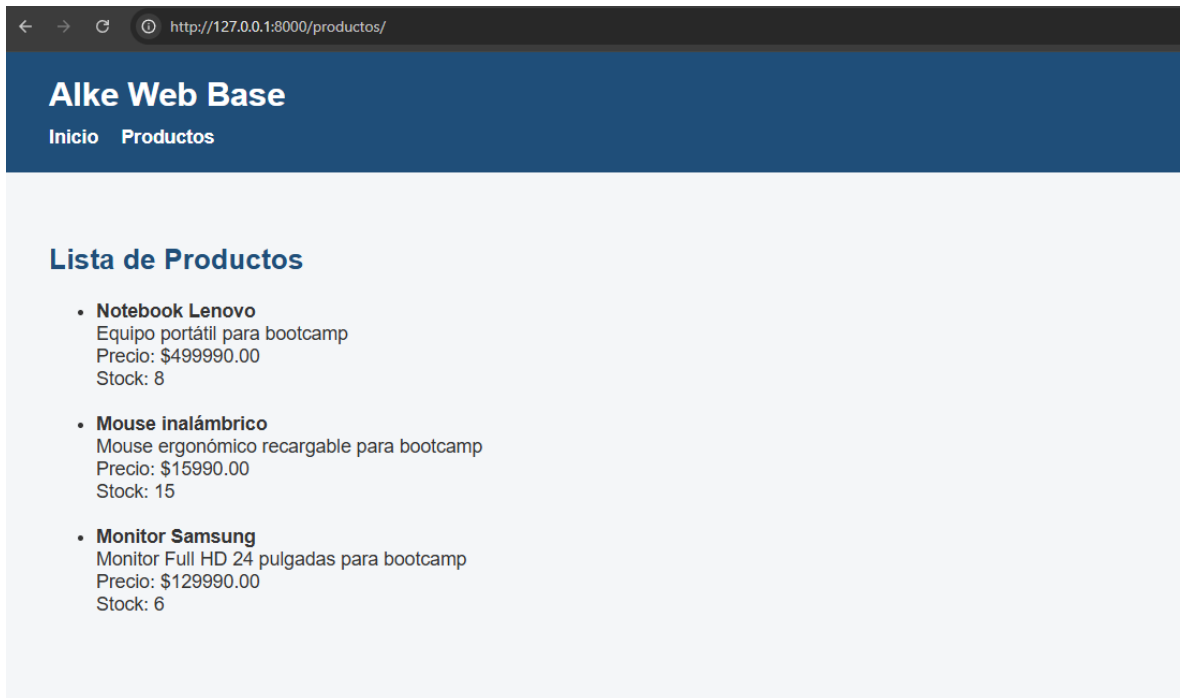
9. Incorporación del enlace de navegación en “base.html”



```
EXPLORADOR
PROYECTO-MODULO-6-DESARROLLO-WEB-DJANGO
  alke_web
    __pycache__
    __init__.py
    asgi.py
    settings.py
    urls.py
    wsgi.py
  venv
  webapp
    __pycache__
    migrations
    static\webapp
      style.css
    templates\webapp
      base.html
      inicio.html
      productos.html
    __init__.py
    admin.py
    apps.py
    models.py
    tests.py
    urls.py
    views.py
  .gitattributes
  db.sqlite3
  manage.py
  README.md

productos.html U
base.html M
webapp > templates > webapp > base.html > html > body > header > nav > a
1 {% load static %}
2 <!DOCTYPE html>
3 <html lang="es">
4
5 <head>
6   <meta charset="UTF-8">
7   <meta name="viewport" content="width=device-width, initial-scale=1.0">
8   <title>{{ block title }}Alke Web Base{% endblock %}</title>
9   <link rel="stylesheet" href="{% static 'webapp/style.css' %}">
10 </head>
11
12 <body>
13   <header>
14     <h1>Alke Web Base</h1>
15     <nav>
16       <a href="{% url 'inicio' %}">Inicio</a>
17       <a href="{% url 'productos' %}">Productos</a>
18     </nav>
19   </header>
20
21   <main>
22     {% block content %}
23     {% endblock %}
24   </main>
25 </body>
26
27 </html>
```

10. Verificación del funcionamiento de /productos/



5.7 Formulario para creación de productos

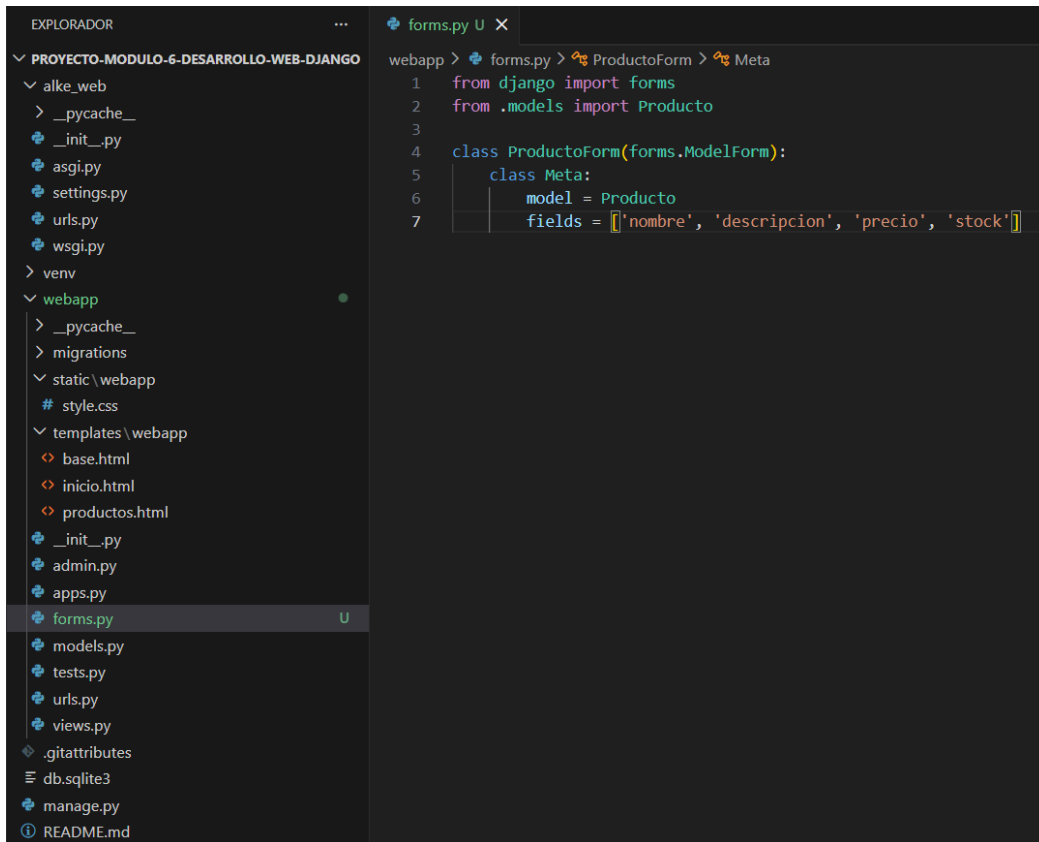
En esta etapa se implementó un formulario web para registrar nuevos productos directamente desde la interfaz del sistema. Para ello, se creó un archivo “forms.py” en la aplicación principal, donde se definió un “ModelForm” llamado “ProductoForm”, vinculado al modelo “Producto”. Este enfoque permitió generar automáticamente un formulario a partir de los campos del modelo y aprovechar el sistema de validación integrado de Django.

Posteriormente, se creó una vista llamada “crear_producto”, encargada de mostrar el formulario cuando la solicitud es de tipo GET y de procesar los datos cuando la solicitud es POST. Si el formulario es válido, el producto se guarda en la base de datos y el usuario es redirigido a la lista de productos.

También se diseñó la plantilla “crear_producto.html”, la cual hereda de “base.html” y muestra el formulario utilizando `{{ form.as_p }}` junto con el token de seguridad `{% csrf_token %}`. Además, se agregó una nueva ruta “/productos/nuevo/” y un enlace de navegación en la plantilla base, permitiendo acceder fácilmente al formulario desde el sitio.

Finalmente, se aplicaron estilos CSS básicos para mejorar la presentación visual del formulario. Con esta implementación, el proyecto ya permite no solo visualizar información dinámica, sino también capturar y almacenar datos desde una interfaz web propia.

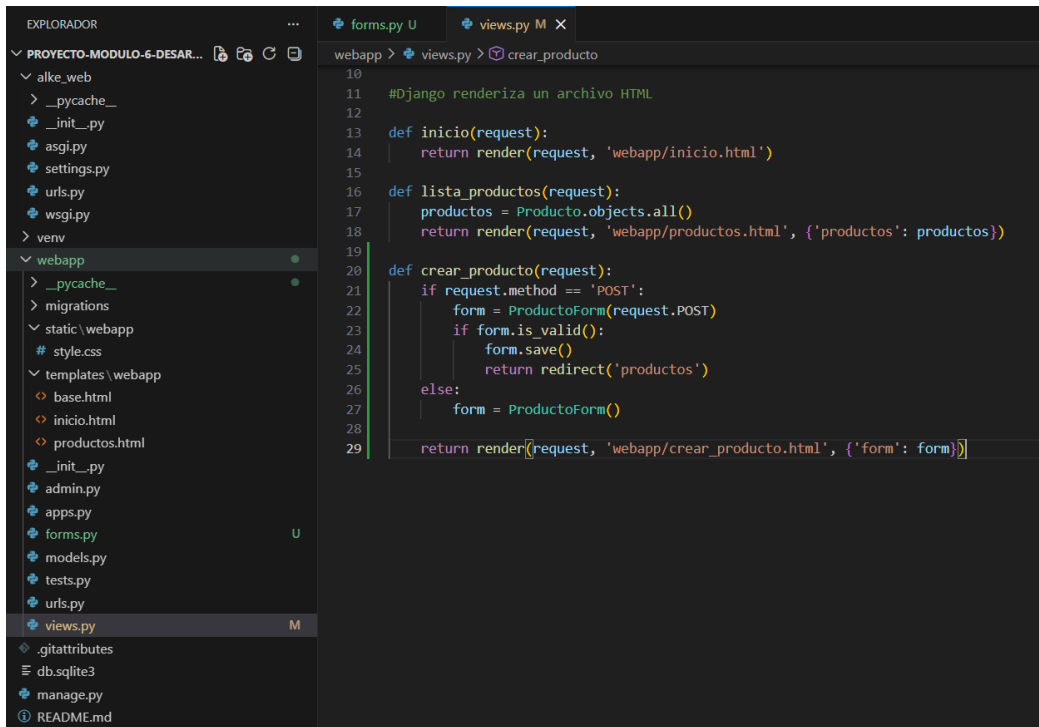
1. Creación “forms.py”



The screenshot shows the VS Code interface with the Explorer on the left and the Editor on the right. The Explorer shows a project structure for 'PROYECTO-MODULO-6-DESARROLLO-WEB-DJANGO' with a subdirectory 'webapp'. The 'forms.py' file is highlighted in the Explorer. The Editor shows the content of 'forms.py' with the following code:

```
webapp > forms.py > ProductoForm > Meta
1 from django import forms
2 from .models import Producto
3
4 class ProductoForm(forms.ModelForm):
5     class Meta:
6         model = Producto
7         fields = ['nombre', 'descripcion', 'precio', 'stock']
```

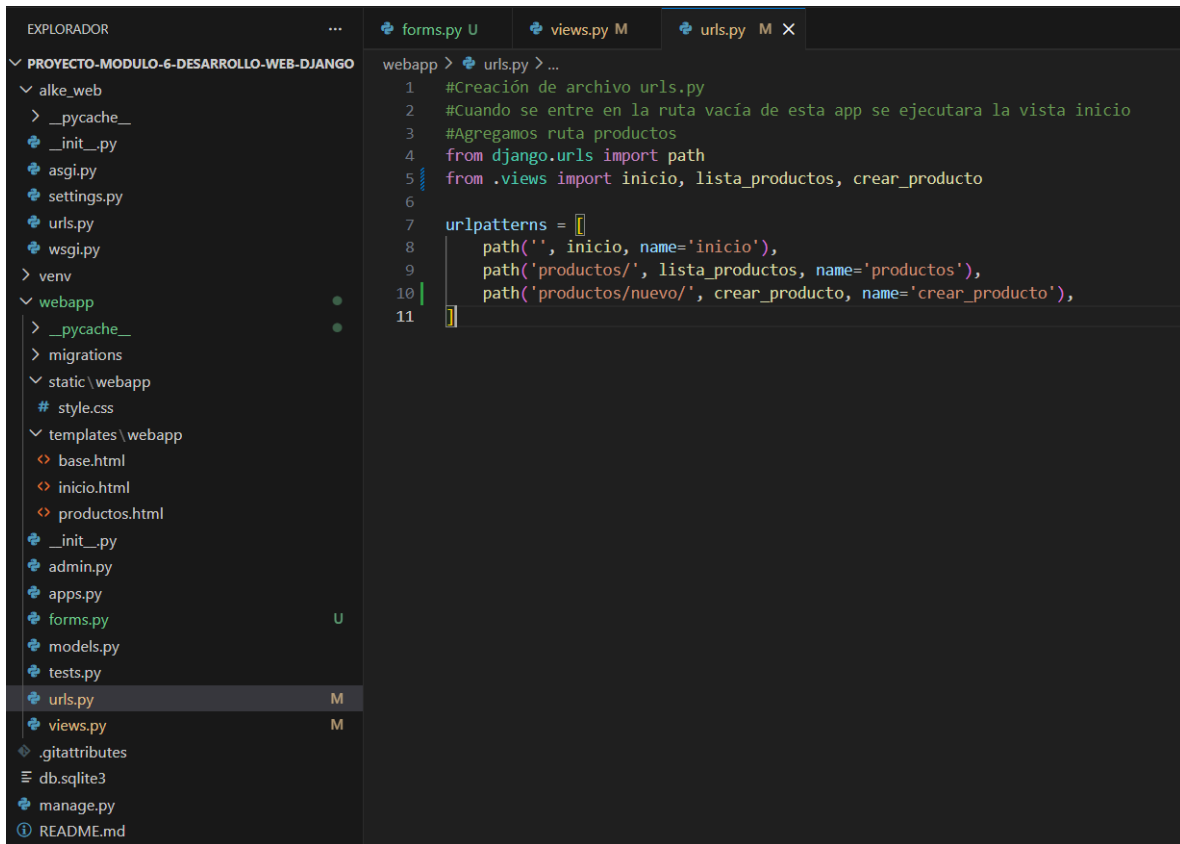
2. Creación de vista para el formulario



The screenshot shows the VS Code interface with the Explorer on the left and the Editor on the right. The Explorer shows the same project structure as before, but now 'views.py' is highlighted. The Editor shows the content of 'views.py' with the following code:

```
webapp > views.py > crear_producto
10
11 #Django renderiza un archivo HTML
12
13 def inicio(request):
14     return render(request, 'webapp/inicio.html')
15
16 def lista_productos(request):
17     productos = Producto.objects.all()
18     return render(request, 'webapp/productos.html', {'productos': productos})
19
20 def crear_producto(request):
21     if request.method == 'POST':
22         form = ProductoForm(request.POST)
23         if form.is_valid():
24             form.save()
25             return redirect('productos')
26     else:
27         form = ProductoForm()
28
29     return render(request, 'webapp/crear_producto.html', {'form': form})
```

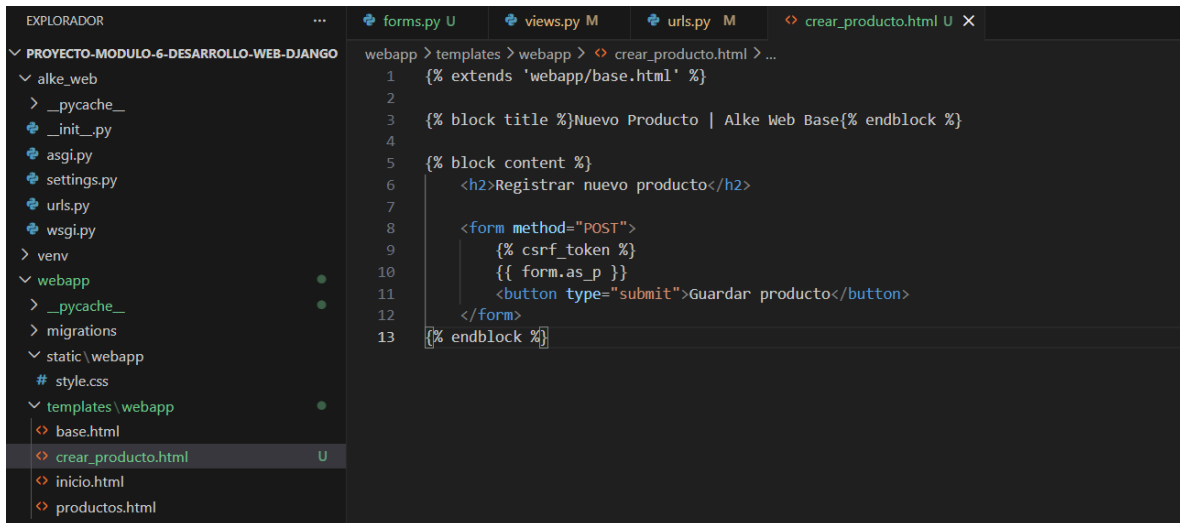
3. Incorporación de nueva ruta



The screenshot shows the VS Code interface with the Django project structure on the left and the `urls.py` file open in the editor. The project structure includes `alke_web`, `webapp`, `static`, and `templates` directories. The `urls.py` file is located in the `webapp` directory. The code in `urls.py` defines the URL patterns for the application.

```
webapp > urls.py > ...
1 #Creación de archivo urls.py
2 #Cuando se entre en la ruta vacía de esta app se ejecutara la vista inicio
3 #Agregamos ruta productos
4 from django.urls import path
5 from .views import inicio, lista_productos, crear_producto
6
7 urlpatterns = [
8     path('', inicio, name='inicio'),
9     path('productos/', lista_productos, name='productos'),
10    path('productos/nuevo/', crear_producto, name='crear_producto'),
11]
```

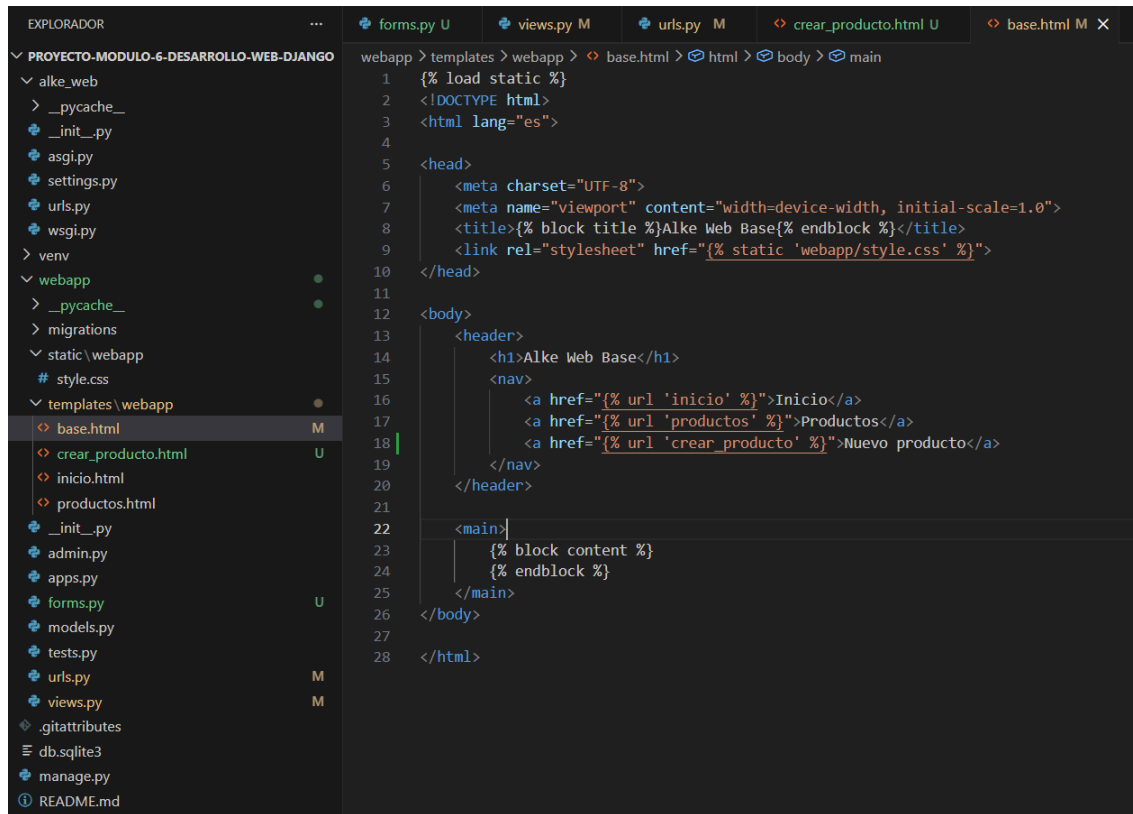
4. Creación del template del formulario



The screenshot shows the VS Code interface with the Django project structure on the left and the `crear_producto.html` template file open in the editor. The project structure is the same as in the previous screenshot. The `crear_producto.html` file is located in the `templates/webapp` directory. The code in `crear_producto.html` defines the HTML structure for the product creation form.

```
webapp > templates > webapp > crear_producto.html > ...
1 {% extends 'webapp/base.html' %}
2
3 {% block title %}Nuevo Producto | Alke Web Base{% endblock %}
4
5 {% block content %}
6     <h2>Registrar nuevo producto</h2>
7
8     <form method="POST">
9         {{ csrf_token }}
10        {{ form.as_p }}
11        <button type="submit">Guardar producto</button>
12    </form>
13 {% endblock %}
```

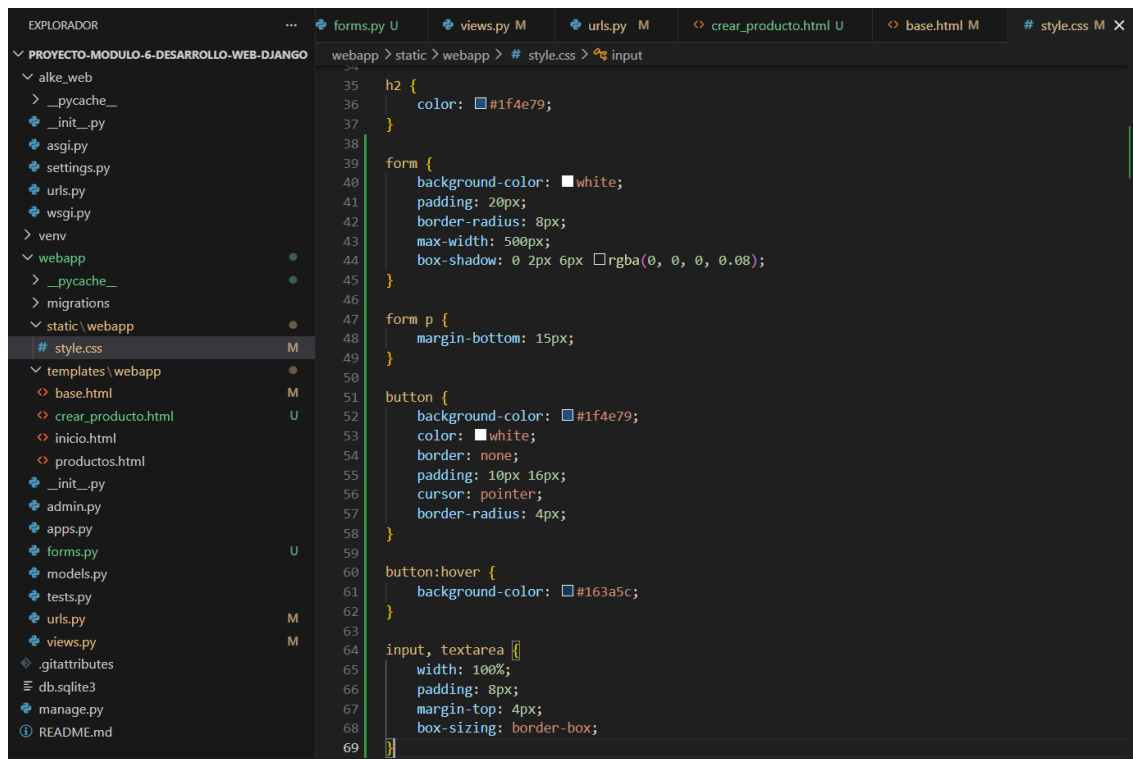
5. Incorporación de enlace en el template “base.html”



The screenshot shows the VS Code interface with the Django project structure on the left and the `base.html` template open in the editor. The project structure includes `PROYECTO-MODULO-6-DESARROLLO-WEB-DJANGO` with subdirectories like `alke_web`, `webapp`, and `static`. The `base.html` file is located at `webapp > templates > webapp > base.html`. The code in the editor is as follows:

```
1 {% load static %}
2 <!DOCTYPE html>
3 <html lang="es">
4
5 <head>
6     <meta charset="UTF-8">
7     <meta name="viewport" content="width=device-width, initial-scale=1.0">
8     <title>{% block title %}Alke Web Base{% endblock %}</title>
9     <link rel="stylesheet" href="{% static 'webapp/style.css' %}">
10 </head>
11
12 <body>
13     <header>
14         <h1>Alke Web Base</h1>
15         <nav>
16             <a href="{% url 'inicio' %}">Inicio</a>
17             <a href="{% url 'productos' %}">Productos</a>
18             <a href="{% url 'crear_producto' %}">Nuevo producto</a>
19         </nav>
20     </header>
21
22     <main>
23         {% block content %}
24         {% endblock %}
25     </main>
26 </body>
27
28 </html>
```

6. Aplicación de estilos en style.css para el formulario



The screenshot shows the VS Code interface with the Django project structure on the left and the `style.css` file open in the editor. The project structure is the same as in the previous screenshot. The `style.css` file is located at `webapp > static > webapp > style.css`. The code in the editor is as follows:

```
35 h2 {
36     color: #1f4e79;
37 }
38
39 form {
40     background-color: white;
41     padding: 20px;
42     border-radius: 8px;
43     max-width: 500px;
44     box-shadow: 0 2px 6px rgba(0, 0, 0, 0.08);
45 }
46
47 form p {
48     margin-bottom: 15px;
49 }
50
51 button {
52     background-color: #1f4e79;
53     color: white;
54     border: none;
55     padding: 10px 16px;
56     cursor: pointer;
57     border-radius: 4px;
58 }
59
60 button:hover {
61     background-color: #163a5c;
62 }
63
64 input, textarea {
65     width: 100%;
66     padding: 8px;
67     margin-top: 4px;
68     box-sizing: border-box;
69 }
```

7. Verificación del funcionamiento del formulario

← → ↻ ⓘ http://127.0.0.1:8000/productos/nuevo/

Alke Web Base

[Inicio](#) [Productos](#) [Nuevo producto](#)

Registrar nuevo producto

Nombre:

Descripcion:

Precio:

Stock:

[Guardar producto](#)

8. Al guardar debe redireccionar a /productos/ con articulo añadido



5.8 Autenticación y autorización

En esta etapa se incorporó el sistema de autenticación de Django para permitir el acceso mediante usuario y contraseña. Para ello, se configuraron en “settings.py” las variables “LOGIN_REDIRECT_URL”, “LOGOUT_REDIRECT_URL” y “LOGIN_URL”, definiendo las redirecciones posteriores al inicio y cierre de sesión, así como la ruta de acceso al formulario de login.

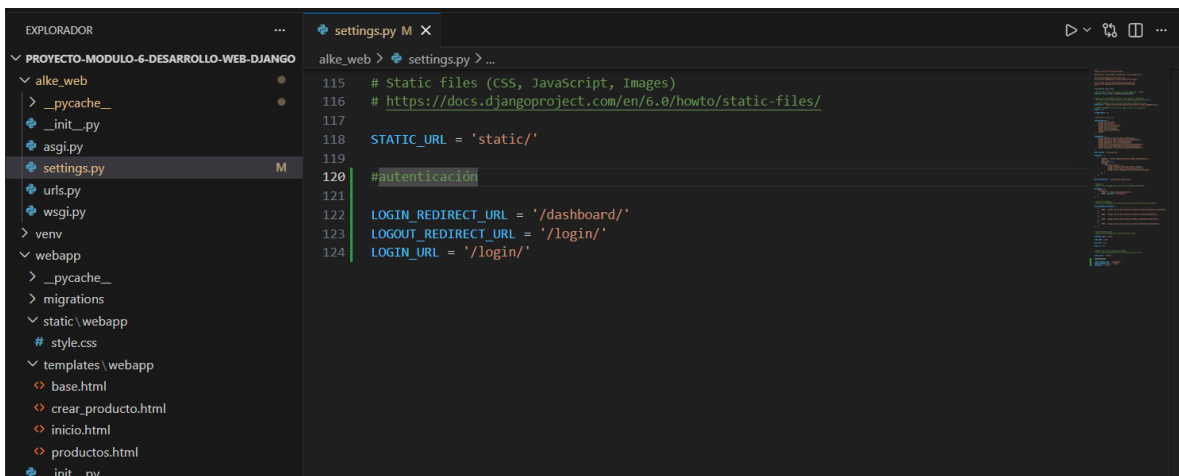
Posteriormente, se agregaron en “urls.py” las rutas correspondientes al login y logout utilizando las vistas predefinidas de autenticación de Django (LoginView y LogoutView). También se creó una vista llamada “dashboard_view”, protegida mediante el decorador “@login_required”, permitiendo que solo los usuarios autenticados pudieran acceder a dicha sección.

Además, se desarrollaron las plantillas “login.html” y “dashboard.html”, ambas integradas con la estructura base del sitio. La plantilla de login incluyó el formulario de autenticación protegido con {% csrf_token %}, mientras que el dashboard mostró un mensaje de bienvenida al usuario autenticado. Finalmente, se actualizó la navegación en “base.html” para mostrar distintas opciones según el estado de autenticación del usuario.

Con esta implementación, el proyecto incorporó un sistema de acceso funcional, permitiendo controlar qué páginas pueden ser vistas solo por usuarios autenticados y mejorando así la seguridad general de la aplicación.

1. Configuración de redirecciones en “settings.py”

- **LOGIN_REDIRECT_URL:** a dónde va el usuario después de iniciar sesión
- **LOGOUT_REDIRECT_URL:** a dónde va después de cerrar sesión
- **LOGIN_URL:** página a la que se redirige si intenta entrar a una vista protegida sin autenticarse

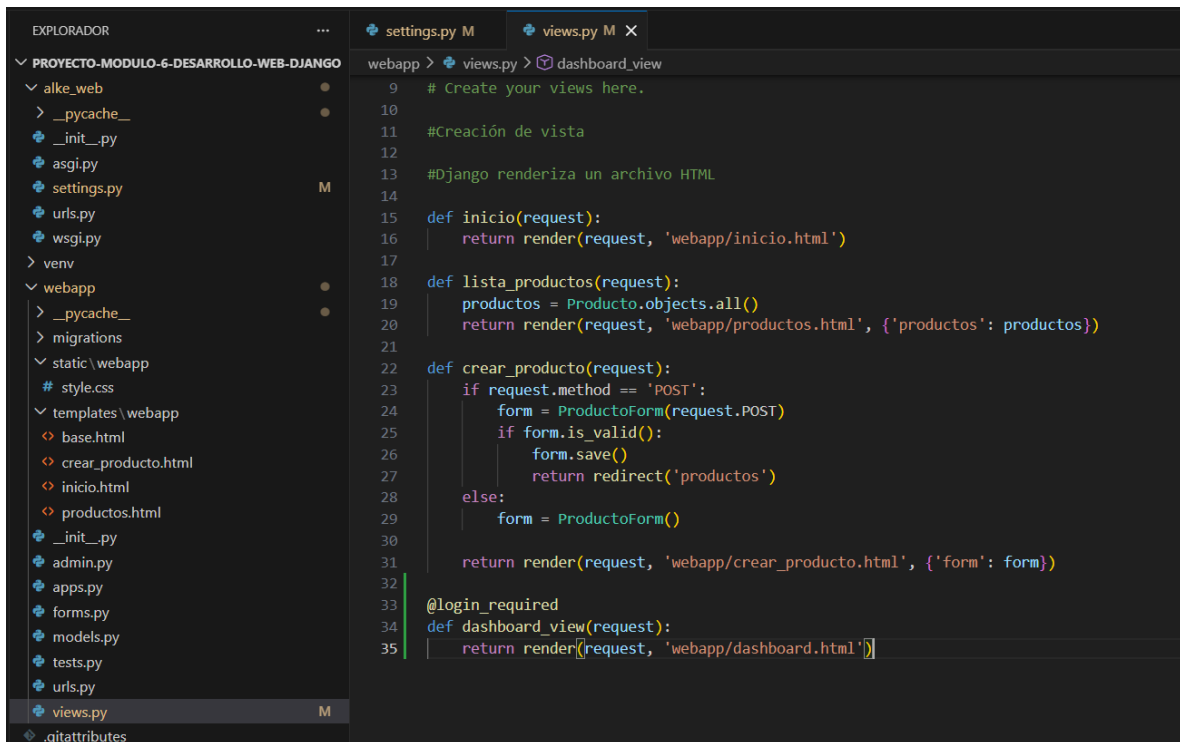


The screenshot shows a code editor with a file explorer on the left and a code editor on the right. The file explorer shows a project structure with folders like 'PROJECTO-MODULO-6-DESARROLLO-WEB-DJANGO', 'alke_web', 'static', and 'templates'. The 'alke_web' folder is expanded, showing files like 'settings.py', 'urls.py', 'wsgi.py', and 'venv'. The 'settings.py' file is selected and open in the code editor. The code in the editor shows the following lines:

```
115 # Static files (CSS, JavaScript, Images)
116 # https://docs.djangoproject.com/en/6.0/howto/static-files/
117
118 STATIC_URL = 'static/'
119
120 #autenticación
121
122 LOGIN_REDIRECT_URL = '/dashboard/'
123 LOGOUT_REDIRECT_URL = '/login/'
124 LOGIN_URL = '/login/'
```

2. Creación de la vista del dashboard

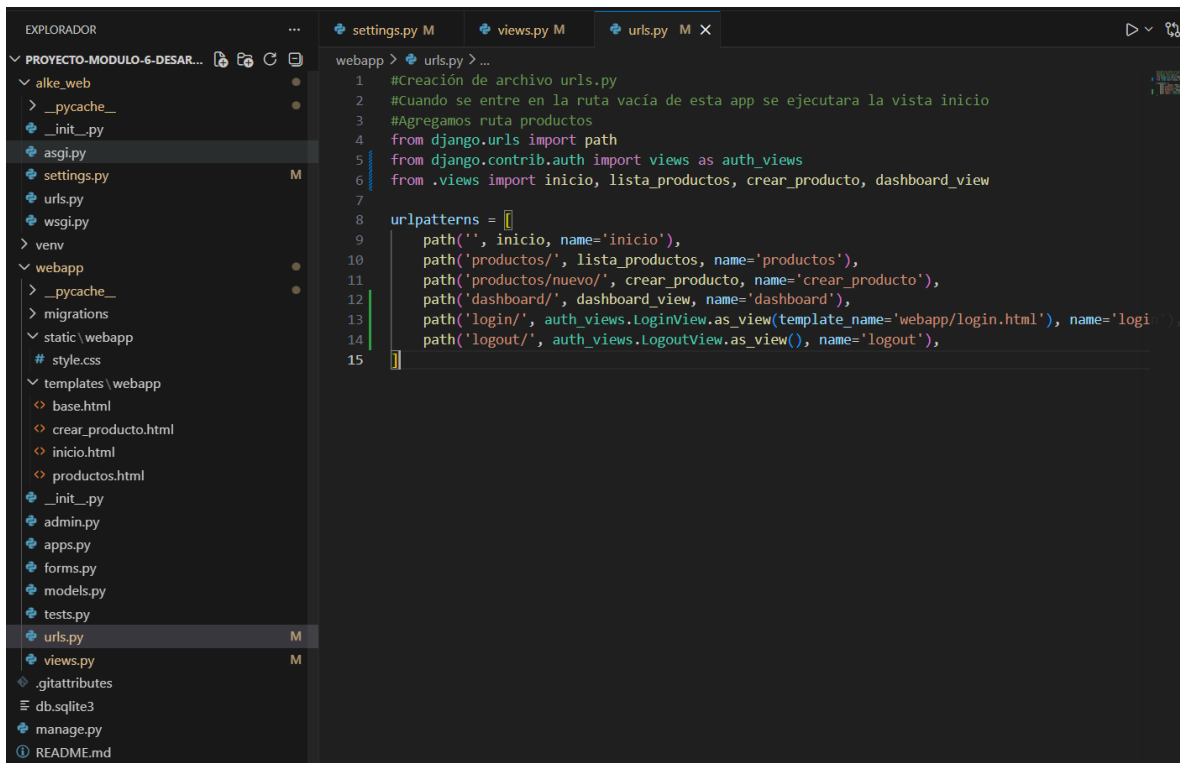
- `@login_required` Protege la vista para que solo un usuario autenticado pueda entrar.



The screenshot shows a code editor with a file explorer on the left and a code editor on the right. The file explorer shows a project structure for 'PROYECTO-MODULO-6-DESARROLLO-WEB-DJANGO'. The code editor shows the 'views.py' file with the following code:

```
9 # Create your views here.
10
11 #Creación de vista
12
13 #Django renderiza un archivo HTML
14
15 def inicio(request):
16     return render(request, 'webapp/inicio.html')
17
18 def lista_productos(request):
19     productos = Producto.objects.all()
20     return render(request, 'webapp/productos.html', {'productos': productos})
21
22 def crear_producto(request):
23     if request.method == 'POST':
24         form = ProductoForm(request.POST)
25         if form.is_valid():
26             form.save()
27             return redirect('productos')
28     else:
29         form = ProductoForm()
30
31     return render(request, 'webapp/crear_producto.html', {'form': form})
32
33 @login_required
34 def dashboard_view(request):
35     return render(request, 'webapp/dashboard.html')
```

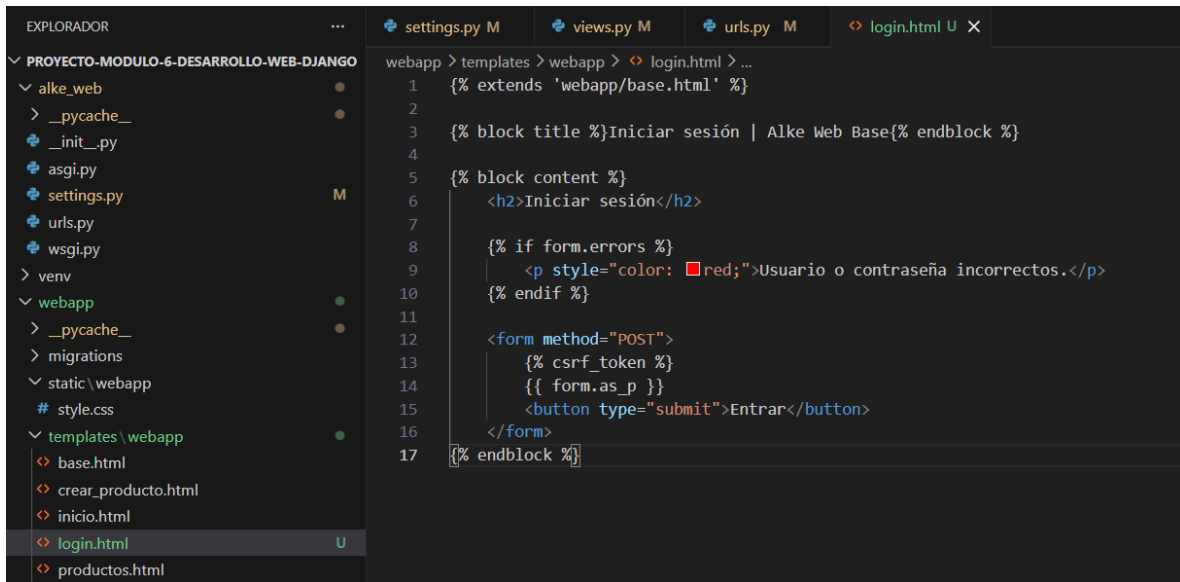
3. Incorporación de rutas del login, logout y dashboard en urls.py



The screenshot shows a code editor with a file explorer on the left and a code editor on the right. The file explorer shows a project structure for 'PROYECTO-MODULO-6-DESAR...'. The code editor shows the 'urls.py' file with the following code:

```
1 #Creación de archivo urls.py
2 #Cuando se entre en la ruta vacía de esta app se ejecutara la vista inicio
3 #Agregamos ruta productos
4 from django.urls import path
5 from django.contrib.auth import views as auth_views
6 from .views import inicio, lista_productos, crear_producto, dashboard_view
7
8 urlpatterns = [
9     path('', inicio, name='inicio'),
10    path('productos/', lista_productos, name='productos'),
11    path('productos/nuevo/', crear_producto, name='crear_producto'),
12    path('dashboard/', dashboard_view, name='dashboard'),
13    path('login/', auth_views.LoginView.as_view(template_name='webapp/login.html'), name='login'),
14    path('logout/', auth_views.LogoutView.as_view(), name='logout'),
15 ]
```

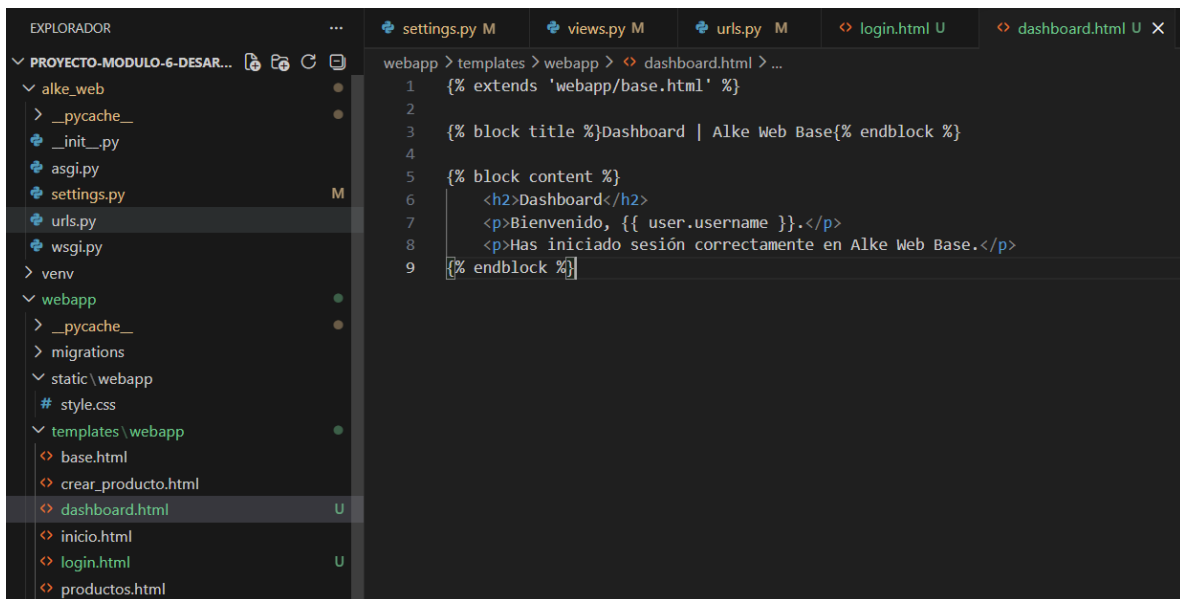
4. Creación del template “login.html”



The screenshot shows the Visual Studio Code interface with the Explorer on the left and the Code editor on the right. The Explorer shows a project structure for 'PROYECTO-MODULO-6-DESARROLLO-WEB-DJANGO' with a 'webapp' folder containing 'templates' and 'webapp' subfolders. The 'login.html' file is selected in the Explorer. The Code editor shows the content of 'login.html' which extends 'webapp/base.html' and contains a login form with a title 'Iniciar sesión | Alke Web Base', a message for incorrect credentials, and a submit button.

```
webapp > templates > webapp > login.html > ...
1 {% extends 'webapp/base.html' %}
2
3 {% block title %}Iniciar sesión | Alke Web Base{% endblock %}
4
5 {% block content %}
6     <h2>Iniciar sesión</h2>
7
8     {% if form.errors %}
9         <p style="color: red;">Usuario o contraseña incorrectos.</p>
10    {% endif %}
11
12    <form method="POST">
13        {% csrf_token %}
14        {{ form.as_p }}
15        <button type="submit">Entrar</button>
16    </form>
17 {% endblock %}
```

5. Creación del template “dashboard.html”

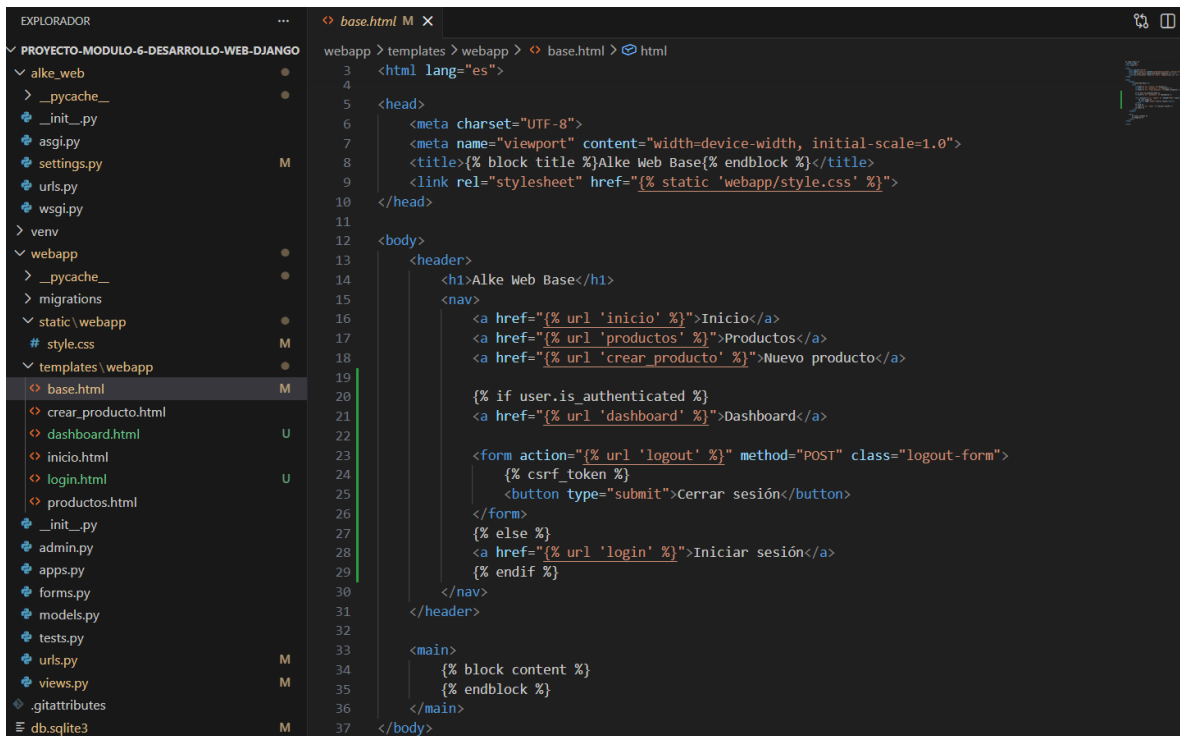


The screenshot shows the Visual Studio Code interface with the Explorer on the left and the Code editor on the right. The Explorer shows the same project structure as the previous screenshot, but now 'dashboard.html' is selected in the Explorer. The Code editor shows the content of 'dashboard.html' which extends 'webapp/base.html' and contains a dashboard page with a title 'Dashboard | Alke Web Base', a welcome message for the user, and a confirmation message for successful login.

```
webapp > templates > webapp > dashboard.html > ...
1 {% extends 'webapp/base.html' %}
2
3 {% block title %}Dashboard | Alke Web Base{% endblock %}
4
5 {% block content %}
6     <h2>Dashboard</h2>
7     <p>Bienvenido, {{ user.username }}.</p>
8     <p>Has iniciado sesión correctamente en Alke Web Base.</p>
9 {% endblock %}
```

6. Actualización del <nav> en “base.html”

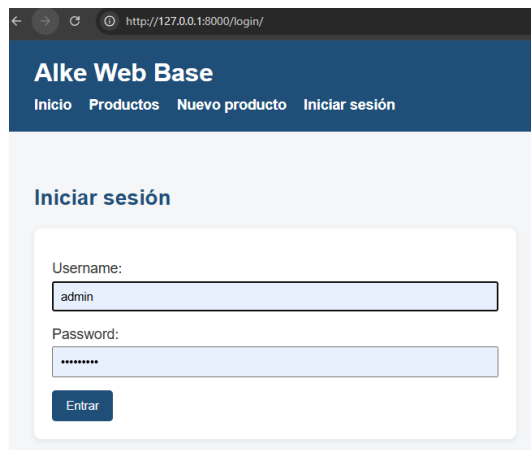
Si el usuario esta autenticado se mostrara el dashboard y el logout



```
webapp > templates > webapp > base.html > html
3 <html lang="es">
4
5 <head>
6   <meta charset="UTF-8">
7   <meta name="viewport" content="width=device-width, initial-scale=1.0">
8   <title>{% block title %}Alke Web Base{% endblock %}</title>
9   <link rel="stylesheet" href="{% static 'webapp/style.css' %}">
10 </head>
11
12 <body>
13   <header>
14     <h1>Alke Web Base</h1>
15     <nav>
16       <a href="{% url 'inicio' %}">Inicio</a>
17       <a href="{% url 'productos' %}">Productos</a>
18       <a href="{% url 'crear_producto' %}">Nuevo producto</a>
19
20       {% if user.is_authenticated %}
21       <a href="{% url 'dashboard' %}">Dashboard</a>
22
23       <form action="{% url 'logout' %}" method="POST" class="logout-form">
24         {{ csrf_token }}
25         <button type="submit">Cerrar sesión</button>
26       </form>
27       {% else %}
28       <a href="{% url 'login' %}">Iniciar sesión</a>
29       {% endif %}
30     </nav>
31   </header>
32
33   <main>
34     {% block content %}
35     {% endblock %}
36   </main>
37 </body>
```

7. Verificación del funcionamiento y flujo del Login

Ejecutamos el servidor con el comando “python manage.py runserver”



Nos dirigimos al login,
<http://127.0.0.1:8000/login/>

Iniciamos sesión en este caso con las credenciales del “superusuario”

Credenciales utilizadas
Username: admin
Password: admin1234

← → ↻ ⓘ http://127.0.0.1:8000/login/

Alke Web Base

[Inicio](#) [Productos](#) [Nuevo producto](#) [Iniciar sesión](#)

Iniciar sesión

Usuario o contraseña incorrectos.

- Please enter a correct username and password. Note that both fields may be case-sensitive.

Username:

Password:

[Entrar](#)

En caso de ingresar credenciales erróneas, esto se indica mediante un mensaje

← → ↻ ⓘ http://127.0.0.1:8000/dashboard/

Alke Web Base

[Inicio](#) [Productos](#) [Nuevo producto](#) [Dashboard](#) [Cerrar sesión](#)

Dashboard

Bienvenido, admin.

Has iniciado sesión correctamente en Alke Web Base.

Verificamos que al iniciar sesión se realice el redireccionamiento a <http://127.0.0.1:8000/dashboard/>

← → ↻ ⓘ http://127.0.0.1:8000/login/

Alke Web Base

[Inicio](#) [Productos](#) [Nuevo producto](#) [Iniciar sesión](#)

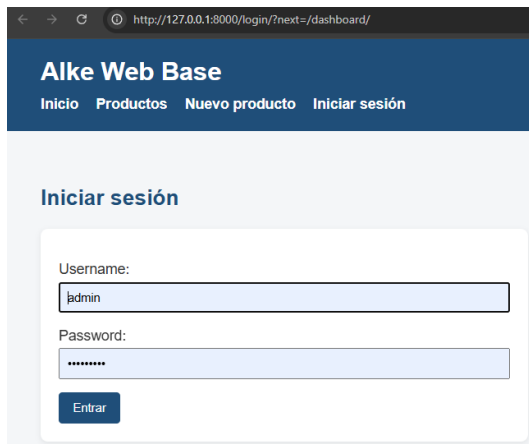
Iniciar sesión

Username:

Password:

[Entrar](#)

Al cerrar sesión nos debiese redireccionar al login, <http://127.0.0.1:8000/login/>



Una vez cerrada la sesión, verificamos vista protegida dashboard, al intentar entrar nos redireccionara a <http://127.0.0.1:8000/dashboard/>

Durante las pruebas del sistema de autenticación, se detectó que la funcionalidad de cierre de sesión generaba un error 405 al intentar ejecutarse mediante una solicitud GET. Para corregirlo, se reemplazó el enlace de logout por un formulario POST con token CSRF, ajustándose así al comportamiento seguro esperado por Django.

6. Resultados obtenidos

El sistema desarrollado permite:

- Visualizar productos almacenados en la base de datos
- Registrar nuevos productos mediante formularios web
- Administrar productos desde el panel administrativo de Django
- Iniciar y cerrar sesión mediante el sistema de autenticación
- Restringir el acceso a ciertas vistas mediante control de usuarios
- Navegar por el sistema mediante una interfaz estructurada y reutilizable

El proyecto cumple con los objetivos planteados, permitiendo demostrar el uso integrado de los principales componentes del framework Django.

7. Conclusión

El desarrollo del proyecto Alke Web Base permitió aplicar de manera práctica los fundamentos del framework Django en la construcción de una aplicación web funcional. A lo largo del proceso se configuró el entorno de desarrollo, se creó la estructura del proyecto y de la aplicación principal, y se implementaron vistas, rutas, templates y archivos estáticos para dar forma a la interfaz del sistema.

Además, se desarrolló un modelo de datos para productos, el cual fue integrado con la base de datos mediante migraciones y utilizado tanto en vistas dinámicas como en formularios web. Esto permitió no solo visualizar información almacenada, sino también registrar nuevos productos directamente desde la aplicación.

Por otra parte, se incorporó un sistema de autenticación con login, logout y vista protegida, fortaleciendo la seguridad del proyecto y permitiendo restringir el acceso a ciertas secciones.

En conclusión, este proyecto permitió comprender el flujo completo de una aplicación Django, desde la configuración inicial hasta la administración de datos y control de acceso, consolidando los aprendizajes del módulo y demostrando la capacidad de construir una aplicación web básica, estructurada y funcional.